CSE 209 — Computer Architecture

Chapter 4 Part 2 · Pipelining: Hazards, Forwarding, Branches, Multiple Issue

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Rifat Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, bit layouts and hand-drawn datapath sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

In timing diagrams: **st** = stall (the instruction is held in its stage while a bubble moves down the pipe); **x** = flushed (the instruction is turned into a bubble).

Contents

Exam Map — What the TF Papers Ask — pipelining questions by year, most-repeated first
Lesson 9 — Pipelining Basics and Performance — 5 stages, latency vs throughput, cycle count, speedup
Lesson 10 — Pipeline Registers, Datapath and Control — IF/ID…MEM/WB contents & sizes, lw/sw walk-through, control
Lesson 11 — Hazards and Stalls without Forwarding — structural, data, control; counting stalls
Lesson 12 — Forwarding and the Double Data Hazard — forwarding conditions, ForwardA/B, forwarding unit
Lesson 13 — Load-Use Hazard, Stalls and Hazard Detection — detection condition, how to stall, HDU design
Lesson 14 — Timing Diagrams and Code Scheduling — cycle counting, A = B + C; D = A + E
Lesson 15 — Branch Hazards — branch in MEM vs ID, flushing, branch data hazards, flush vs stall, jumps
Lesson 16 — Branch Prediction — static/dynamic, 1-bit vs 2-bit, branch target buffer, delayed branch
Lesson 17 — Multiple Issue, Scheduling and Loop Unrolling — ILP, VLIW vs superscalar, dual-issue IPC, speculation
Lesson 18 — Exceptions in the Pipeline (not in the slide file) — short answer kit for the recent exception questions
Quick Notes — All Lessons — merged revision notes + key diagrams

> **NOTE** Slide numbers refer to **Chapter 4 Part 2** (99 slides). "TF" questions are quoted from the TF collection and labelled by **session and course code**. Each lesson goes **worked example → rules → exam approach → practice with solutions**. Practice provenance: **Actual** = taken from a TF paper; **Adapted** = TF question with changed details; **Constructed** = new question in TF style.

Exam Map — What the TF Papers Ask

Pipelining appears in **every** paper in the collection. The newest five sessions are the priority; repeats from older papers show which question types are "favourites".

Session (paper)	Pipelining questions
2023-24 (CSE 209)	Q1(a) design a complete **forwarding unit** with double data hazard · Q1(b) **stalls/total cycles with two branches** (branch in ID, forwarding, load-use = 1 stall) · Q1(c) **1-bit predictor** nested-loop problem + **"perfect predictor, 1-cycle penalty"** · Q4(b) **dual-issue scheduling**, IPC, technique to raise IPC
2022-23 (CSE 209)	Q1(c) **A = B + C; D = A + E** cycles + code scheduling · Q2 identify hazards, detection conditions, complete the **hazard detection unit** · Q5 can **flush and stall** occur together? · Q7 **jump stall** (Chandler/Joey/Ross) · Q8 **precise exceptions** in the pipelined datapath
2022-23 (CSE 305)	Q1(a) pipeline **without forwarding** + total cycles · Q1(b) with forwarding: pipeline registers, forwarding lines, **double data hazard** · Q1(c) forwarding condition · Q4(b) **four ID-stage components** that reduce branch stalls
2021-22 (CSE 305)	Q1(b) lw/lw/add/sw… **cycles + code scheduling** · Q1(c) **data hazards for branches** (0/1/2 stalls) · Q1(d) **1-bit predictor** nested loop · Q2(a) **forwarding unit** with double data hazard · Q2(b) **dynamically scheduled superscalar** block diagram · Q2(c) **dual-issue scheduling**, IPC
2020-21 (CSE 305)	Q4 identify data hazards on a multi-cycle diagram, **forwarding conditions**, can forwarding solve all? · Q5 **multiple issue**: refactor a loop to maximise IPC · Q1 "code reordering" ↔ eight great ideas
2019-20 (CSE 305)	Q5 **A = B + C; D = A + D** pipeline with stalls and forwarding paths, stalls/cycles, rearrangement · Q8(b) **2-bit predictor FSM** · Q8(c) **forwarding conditions** sub→and, sub→or
2018-19	Q2(a) **flush and stall** together (branch in MEM) · Q2(b) lw/add/beq with early branch — cycles · Q2(c) TLB-miss exception
2017-18	Q1(a) **A = B + C; D = A + E** · Q1(c) **speedup** pipelined vs single-cycle (3 lw, then +3,000,000) · Q1(d) **perfect predictor** penalty · Q2(a) double data hazard, load-use detection, how to stall · Q2(b) **forwarding unit** · Q2(c) branch data hazards · Q2(d) **1-bit predictor**
Older papers	speedup (2015-16, 2013-14) · hazard types in the slide-22 code (2016-17, 2014-15) · insert pipeline registers/forwarding/HDU into single-cycle datapath (2016-17, 2014-15) · dynamic prediction techniques (2014-15) · branch data hazards + 1-bit (2013-14) · forwarding unit, when forwarding fails (2012-13) · ID/EX contents, double hazard example, **2-bit predictor accuracy** (older paper, TF p. 85)

EXAM FOCUS **Most repeated, in order:** (1) forwarding conditions / forwarding unit with double data hazard · (2) cycle counting + code scheduling (A = B + C; D = A + E family) · (3) 1-bit predictor nested loops & "perfect predictor" penalty · (4) data hazards for branches (0/1/2 stalls) · (5) load-use detection + how to stall · (6) dual-issue scheduling and IPC · (7) flush-vs-stall and jump-stall reasoning.

Lesson 9 — Pipelining Basics and Performance

Slides: Chapter 4 Part 2, 2–9 and 32

EXAM FOCUS Speedup of pipelined over single-cycle for a few lw instructions, then for millions more, is asked in **2017-18 Q1(c)** and older papers (2015-16, 2013-14). The "latency vs throughput" idea is needed in almost every pipelining answer.

1. The Idea: Overlap the Work

Laundry analogy (slide 2): washing, drying, folding and storing are separate steps. Instead of finishing one load completely before starting the next, start the next wash as soon as the washer is free. Each load still takes the same time, but loads **finish more often**.

A MIPS instruction is split the same way into **five stages, one step per stage** (slide 3):

Stage	Work done
IF	Instruction fetch from memory (and PC + 4)
ID	Instruction decode & register read
EX	Execute the operation or calculate an address
MEM	Access the data-memory operand
WB	Write the result back to a register

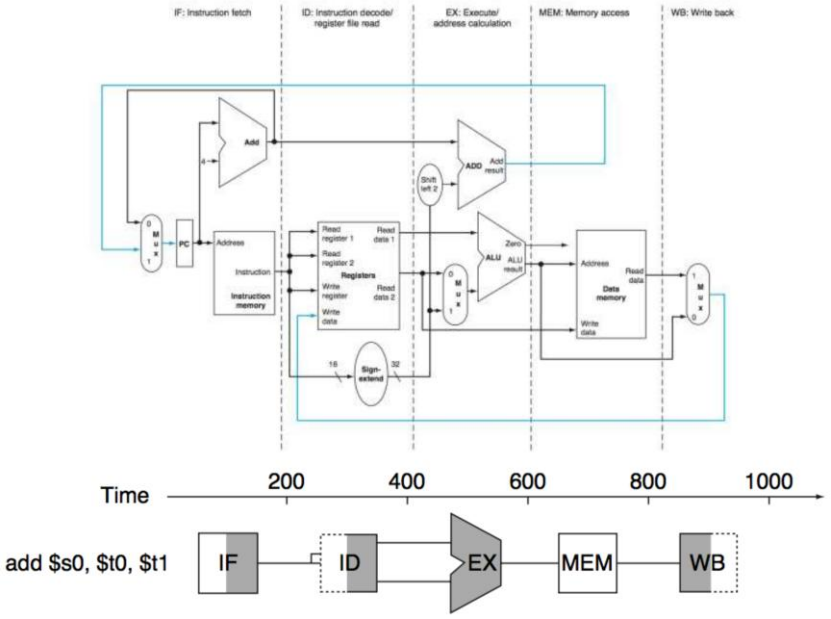


Figure 9.1 — The five MIPS stages laid over the datapath; add $s0,$t0,$t1 uses IF-ID-EX-MEM-WB (slide 4)

2. Worked Example: Single-Cycle vs Pipelined Time

Assume the stage times of slide 5: **100 ps** for register read or write, **200 ps** for the other stages.

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total
lw	200	100	200	200	100	**800 ps**
sw	200	100	200	200	—	700 ps
R-format	200	100	200	—	100	600 ps
beq	200	100	200	—	—	500 ps

Step 1 — Clock periods

- **Single-cycle:** the clock must fit the slowest instruction → T_c **= 800 ps**.
- **Pipelined:** the clock must fit the slowest **stage** → T_c **= 200 ps**. The 100 ps register stages still get a full 200 ps slot.

Step 2 — Time for three lw instructions

```
              1   2   3   4   5   6   7
lw $1,100($0) IF  ID  EX  MEM WB
lw $2,200($0)     IF  ID  EX  MEM WB
lw $3,300($0)         IF  ID  EX  MEM WB
```

- **Single-cycle:** 3 × 800 = **2400 ps**.
- **Pipelined:** the first lw needs 5 cycles; each later one finishes 1 cycle after the previous → (5 + 2) × 200 = **1400 ps**.

Speedup = 2400 / 1400 = **1.714**

Step 3 — Add 1,000,000 more instructions (slide 8)

$T_{pipelined}$ = 1,000,000 × 200 + 1400 = 200,001,400 ps

$T_{nonpipelined}$ = 1,000,000 × 800 + 2400 = 800,002,400 ps

Speedup = 800,002,400 / 200,001,400 ≈ **4**

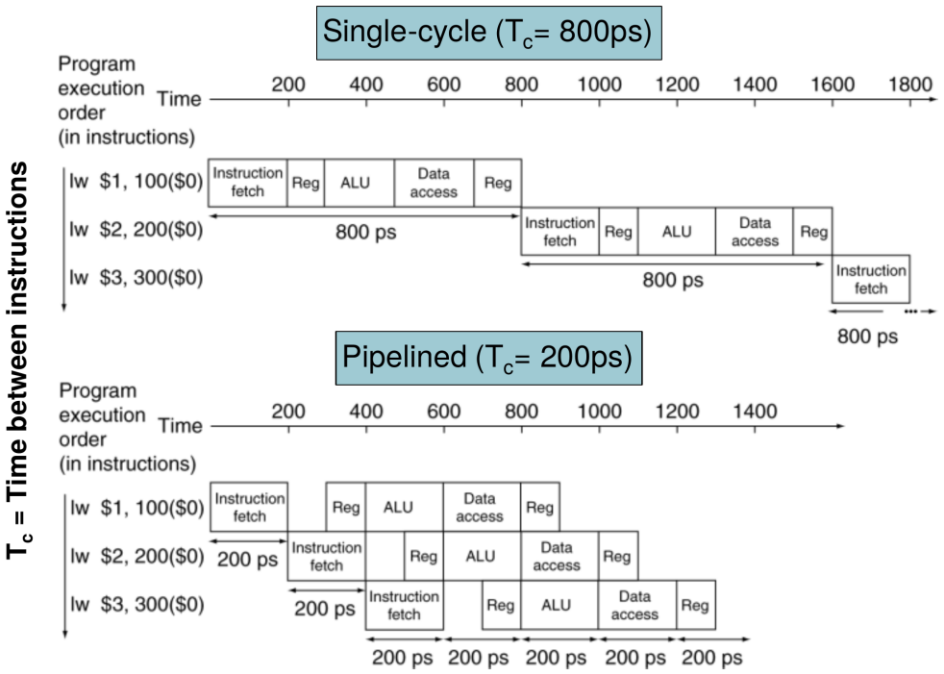


Figure 9.2 — Single-cycle (T_c = 800 ps) versus pipelined (T_c = 200 ps) execution of three loads (slide 6)

MEMORIZE With many instructions the speedup approaches **(single-cycle clock) / (pipelined clock)** = 800 / 200 = 4. It is **not 5**, because the stages are **unbalanced** (balanced stages would give 800 / 5 = 160 ps).

3. Rules and Formulas

Pipelined cycles = k + (n − 1) + stalls (k = number of stages = 5, n = instructions)

Pipelined time = cycles × T_{stage} Single-cycle time = n × T_{single}

$$\text{Time between instructions}_{\text{pipelined}} = \text{Time between instructions}_{\text{nonpipelined}} / \text{Number of stages} \quad \text{(balanced stages only)}$$

- **Throughput improves; latency does not.** Each instruction still passes through all five stages. In the example a lw takes 5 × 200 = 1000 ps pipelined — even longer than 800 ps single-cycle.
- If stages are **not balanced**, speedup is less than the number of stages.
- Speedup comes from **increased throughput** (one instruction completes every cycle once the pipe is full).

Why the MIPS ISA suits pipelining (slide 9)

- **All instructions are 32 bits** → easy to fetch and decode in one cycle (x86: 1–17 bytes).
- **Few, regular formats** → can decode and read registers in one step.
- **Load/store addressing** → address calculated in the 3rd stage, memory accessed in the 4th.
- **Aligned memory operands** → a memory access takes only one cycle.

Exam Approach

1. Single-cycle clock = **sum of all stage delays of the longest instruction** (usually lw).
2. Pipelined clock = **largest single stage delay**.
3. Pipelined cycles for n instructions = **n + 4** (plus stalls); multiply by the pipelined clock.
4. Speedup = single-cycle time ÷ pipelined time. For huge n, speedup → $T_{single} \div T_{stage}$.

TRAP

- Use the delays **given in the question**, not the slide's 800/200 ps (2017-18 uses ALU = 100 ps, so single-cycle lw = 700 ps).
- Do not forget the **fill time**: n instructions need n + 4 cycles, not n.
- "Pipelining reduces the time of each instruction" is **false** — it reduces the time **between** instructions.

Practice — Lesson 9

Part (a): Actual — 2017-18 Q1(c). Part (b): Actual — 2022-23 (CSE 305) Q6(a). Part (c): Constructed.

(a) The stage times for lw are: instruction fetch 200 ps, register read 100 ps, ALU op 100 ps, memory access 200 ps, register write 100 ps. For the program lw $1,100($0); lw $2,200($0); lw $3,300($0), what is the speedup of the pipelined datapath over the single-cycle datapath? What will the speedup be if we add 3,000,000 more lw instructions?

(b) "Joey claims that the number of cycles is identical to the number of instructions." Do you agree?

(c) In the slide-5 example, what is the latency of one lw in the pipelined design, and why is it larger than in the single-cycle design?

✓ Solution

(a) Single-cycle clock = 200 + 100 + 100 + 200 + 100 = **700 ps**. Pipelined clock = largest stage = **200 ps**.

$$\text{3 lw: } T_{single} = 3 \times 700 = 2100 \text{ ps} \qquad T_{pipe} = (5 + 2) \times 200 = 1400 \text{ ps}$$
$$\text{Speedup} = 2100 / 1400 = \mathbf{1.5}$$

With 3,000,000 more (n = 3,000,003):

$$T_{single} = 3{,}000{,}003 \times 700 = 2{,}100{,}002{,}100 \text{ ps}$$
$$T_{pipe} = (3{,}000{,}003 + 4) \times 200 = 600{,}001{,}400 \text{ ps}$$
$$\text{Speedup} = 2{,}100{,}002{,}100 / 600{,}001{,}400 \approx \mathbf{3.5} \quad (\rightarrow 700 / 200)$$

(b) Disagree in general. Cycles = instruction count × CPI, so they are equal only when CPI = 1. Even an ideal 5-stage pipeline needs **n + 4** cycles (4 cycles to fill the pipe), and hazards add stall cycles. Instructions that take several cycles (or a multicycle implementation) make cycles larger still. The two numbers become *approximately* equal only for a long program on an ideal pipeline.

(c) Latency = 5 stages × 200 ps = **1000 ps** (vs 800 ps single-cycle). Every stage is stretched to the slowest stage time (the 100 ps register stages waste 100 ps each), and pipeline registers would add a little more in real hardware. Pipelining improves **throughput**, not latency.

Lesson 10 — Pipeline Registers, Datapath and Control

Slides: Chapter 4 Part 2, 33–52

EXAM FOCUS "What values are stored in the IF/ID and ID/EX pipeline registers?" (older paper, TF p. 85), "draw the pipelined datapath and control for lw" (2015-16, 2013-14), "insert pipeline registers into the single-cycle datapath" (2016-17, 2014-15). The forwarding and hazard questions of later lessons all use these register names.

1. From Single-Cycle to Pipelined Datapath

Cut the single-cycle datapath into the five stages. Information flows **left to right**, with two exceptions (slide 33):

- **WB → register file** (write-back goes back to ID) → the source of **data hazards**.
- **MEM → PC** (branch decision selects the next PC) → the source of **control hazards**.

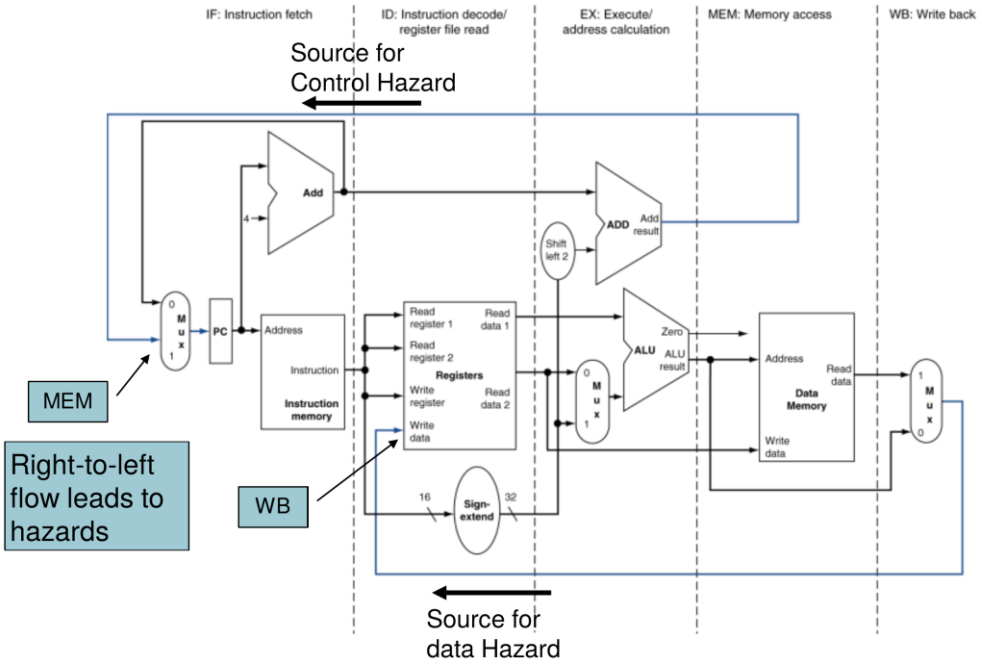


Figure 10.1 — MIPS pipelined datapath; the two right-to-left flows cause hazards (slide 33)

2. Pipeline Registers

Each stage needs to **hold what it produced** so the next stage can use it in the next cycle. Four registers sit between the stages: **IF/ID, ID/EX, EX/MEM, MEM/WB** (named after the two stages they separate).

Register	Holds (basic datapath, slide 34)	Size
IF/ID	Instruction (32) + PC + 4 (32)	32 + 32 = **64**
ID/EX	PC + 4, Read data 1, Read data 2, sign-extended immediate	32 × 4 = **128**
EX/MEM	Branch target, Zero (1 bit), ALU result, Read data 2 (store data)	32 × 3 + 1 = **97**
MEM/WB	Memory read data, ALU result	32 + 32 = **64**

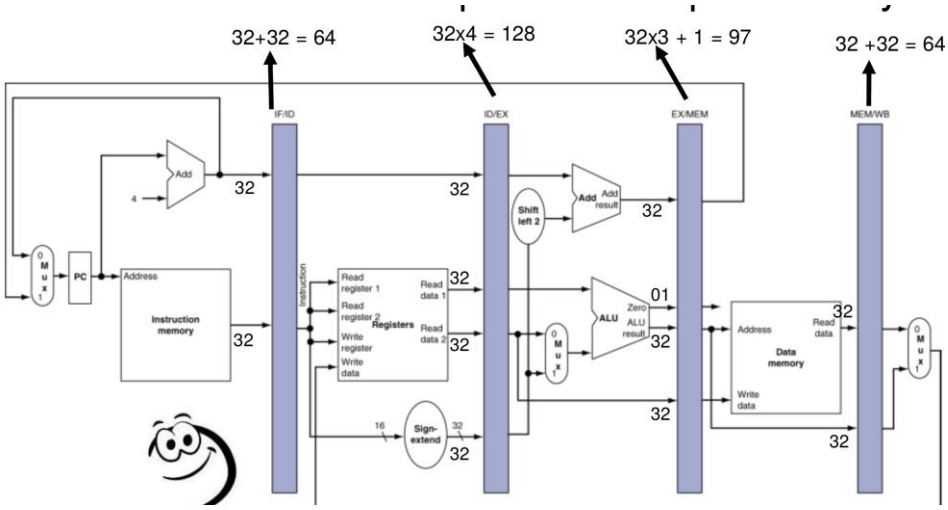


Figure 10.2 — Pipeline registers and their basic sizes (slide 34)

> **NOTE** The slide warns "we may need to update the sizes as we proceed". After the corrections below, the registers also carry **register numbers** and **control bits**: ID/EX + rt, rd (10 bits) + 9 control bits = **147** (+ rs, 5 bits, once forwarding is added = 152); EX/MEM + destination register number (5) + 5 control bits = **107**; MEM/WB + destination register number (5) + 2 control bits = **71**. State your assumptions if you give the larger sizes.

3. Worked Example: lw Walks Through the Pipeline

```
lw $t0, 20($t1)
```

Stage	Work	Saved into
IF	Read instruction at PC; compute PC + 4	IF/ID: instruction, PC + 4
ID	Read $t1 (and rt); sign-extend 20	ID/EX: PC + 4, R[$t1], R[rt], 20 (32-bit)
EX	ALU: R[$t1] + 20	EX/MEM: ALU result (address), …
MEM	Read data memory at that address	MEM/WB: read data
WB	Write the read data into $t0	(register file)

The bug: wrong register number in WB (slides 44–45)

In WB the register file's **Write register** input is wired to the instruction **currently in IF/ID** — a later instruction! So lw would write into the wrong register.

Fix: carry the **destination register number** along with the instruction: ID/EX → EX/MEM → MEM/WB → Write register.

> **MEMORIZE** **Rule:** anything an instruction needs in a later stage must **travel with it** in the pipeline registers — data values, register numbers and control signals.

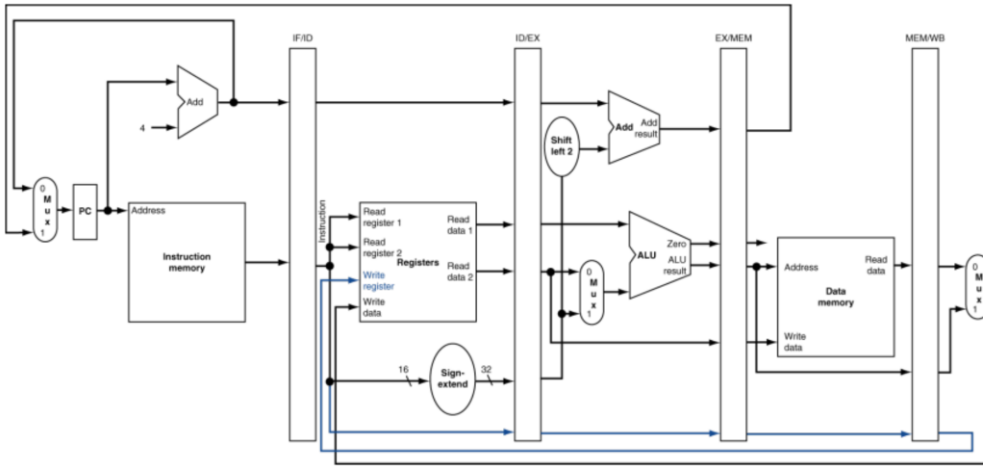


Figure 10.3 — Corrected datapath for load: the write-register number travels to WB (slide 45)

Store (slides 41–43)

sw computes its address in EX, writes memory in MEM, and **does nothing in WB**. Its store data (Read data 2) must be carried through ID/EX and EX/MEM.

4. Two Kinds of Pipeline Diagram (slides 46–49)

- **Multi-cycle pipeline diagram:** time across, instructions down — shows the whole program over time. Use this for hazards and cycle counting.
- **Single-cycle pipeline diagram:** a snapshot of the datapath in **one** clock cycle, showing which instruction is in each stage.

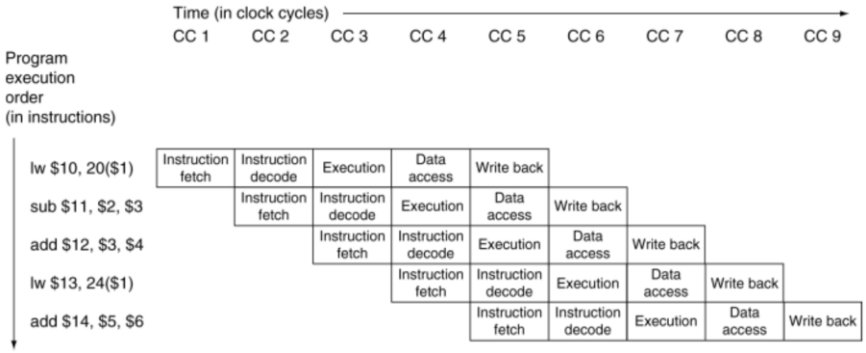


Figure 10.4 — Multi-cycle pipeline diagram, traditional form (slide 47)

Example (slide 49): in **cycle 5** of the sequence lw $10,20($1); sub $11,$2,$3; add $12,$3,$4; lw $13,24($1); add $14,$5,$6 → IF: add $14 · ID: lw $13 · EX: add $12 · MEM: sub $11 · WB: lw $10.

5. Pipelined Control (slides 50–52)

The control signals are the **same as in the single-cycle design**. They are generated in **ID** from the opcode and then **travel in the pipeline registers** to the stage that uses them.

- **IF and ID** need no special control (PC and IF/ID are written every cycle; registers are read every cycle).
- **EX:** RegDst, ALUOp1, ALUOp0, ALUSrc
- **MEM:** Branch, MemRead, MemWrite
- **WB:** RegWrite, MemtoReg

Instruction	RegDst	ALUOp 1	ALUOp 0	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg
R-format	1	1	0	0	0	0	0	1	0
lw	0	0	0	1	0	1	0	1	1
sw	X	0	0	1	0	0	1	0	X
beq	X	0	1	0	1	0	0	0	X

ID/EX carries **WB + M + EX** fields, EX/MEM carries **WB + M**, MEM/WB carries **WB** only.

TRAP RegWrite is *generated* in ID but **used in WB** — it must travel with the instruction (slide 50: "Confusing!!! RegWrite is actually at WB stage"). Using the current instruction's RegWrite in WB would be the same bug as the wrong register number.

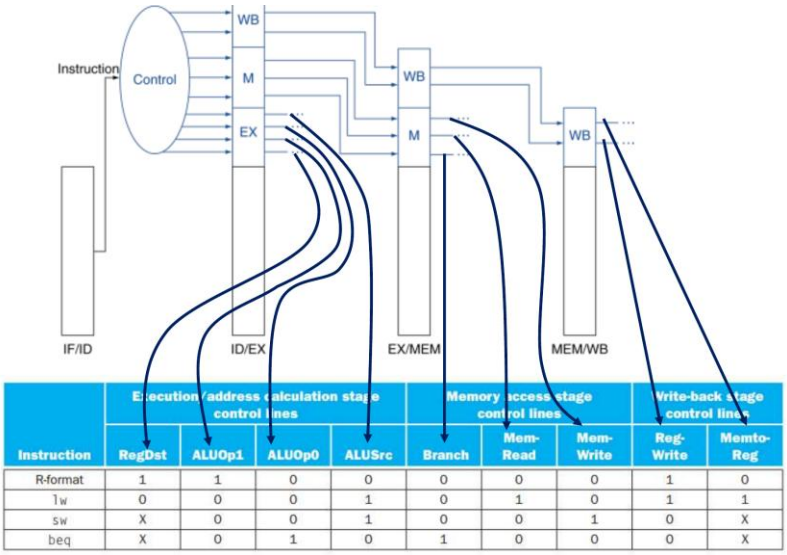


Figure 10.5 — Control signals grouped by the stage that uses them (slide 51)

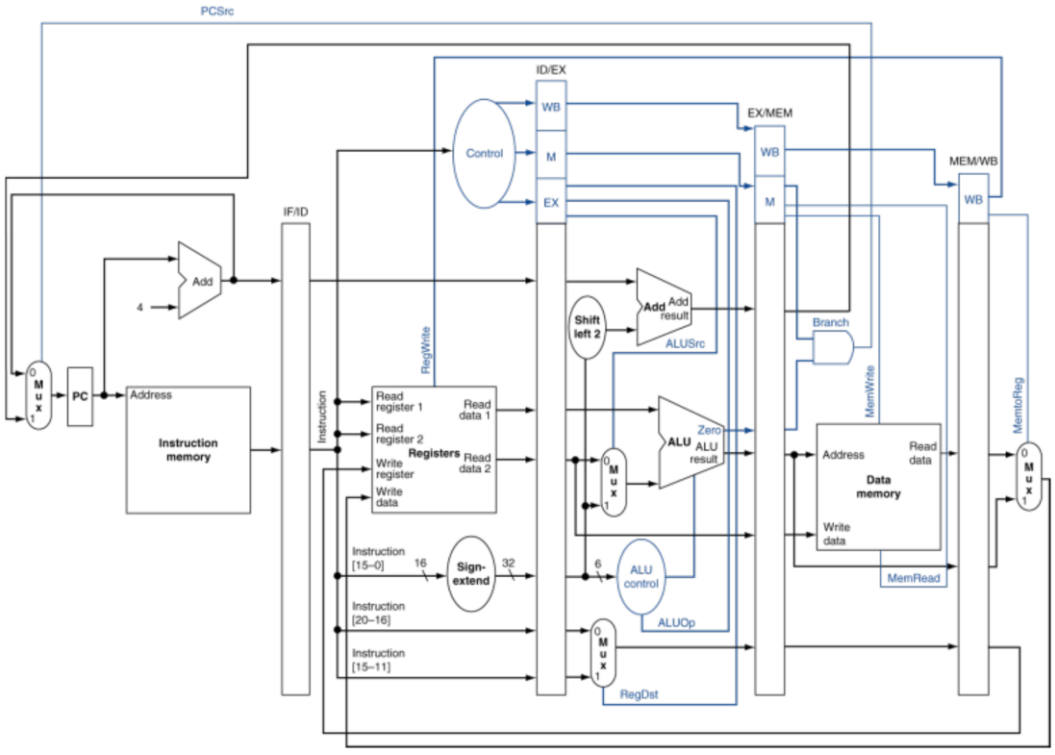


Figure 10.6 — Pipelined datapath with control (slide 52)

Practice — Lesson 10

Part (a): Actual (older paper, TF p. 85). Parts (b)–(d): Constructed.

(a) What values are stored in the IF/ID and ID/EX pipeline registers?

(b) Why must the destination register number of a lw be carried in ID/EX, EX/MEM and MEM/WB?

(c) Which control signals are stored in EX/MEM, and why are the EX-stage signals not stored there?

(d) For lw $10,20($1); sub $11,$2,$3; add $12,$3,$4; lw $13,24($1); add $14,$5,$6, which instruction is in each stage during cycle 4?

✓ Solution

(a) IF/ID: the fetched instruction (32 bits) and PC + 4 (32 bits) — 64 bits. **ID/EX:** PC + 4, Read data 1, Read data 2 and the sign-extended immediate (4 × 32 = 128 bits), plus the register numbers rt and rd (and rs with forwarding) and the WB/M/EX control bits.

(b) The register write happens in WB, four cycles after the instruction was decoded. By then IF/ID holds a different (later) instruction, so its rt/rd field would name the **wrong register**. Carrying the number with the instruction makes WB write the correct register.

(c) EX/MEM stores the **M** signals (Branch, MemRead, MemWrite) and the **WB** signals (RegWrite, MemtoReg). The EX signals (RegDst, ALUOp, ALUSrc) were already used in the EX stage, so they are dropped.

(d) Cycle 4: IF — lw $13,24($1); ID — add $12,$3,$4; EX — sub $11,$2,$3; MEM — lw $10,20($1); WB — empty.

Lesson 11 — Hazards and Stalls without Forwarding

Slides: Chapter 4 Part 2, 10–15 and 25

EXAM FOCUS **2022-23 (CSE 305) Q1(a):** build the pipeline **without forwarding** and count total cycles. Older papers (2016-17, 2014-15) ask "what type of hazard occurs in this code, and how can it be overcome?" and "explain a structural hazard with a time-space diagram".

1. What Is a Hazard?

A **hazard** is a situation that prevents the next instruction from starting in the next cycle (slide 10).

Type	Cause	MIPS example / cure
Structural	A required resource is busy	One memory for both instructions and data → use **separate instruction/data memories (caches)**
Data	An instruction needs a result a previous instruction has not written yet	add $s0,… then sub …,$s0,… → **forwarding**, stalls, code scheduling
Control	The next fetch depends on a branch that is not decided yet	beq → decide early, **predict**, flush (Lessons 15–16)

2. Structural Hazard (slide 11)

With a **single memory**, a load/store accesses data in MEM while a later instruction wants to fetch in IF — in the same cycle. Fetch must stall, creating a **bubble**.

```
                    1   2   3   4   5   6   7   8   9
lw  $1,0($2)        IF  ID  EX  MEM WB
Instr 2                 IF  ID  EX  MEM WB
Instr 3                     IF  ID  EX  MEM WB
Instr 4 (1 memory)              st  IF  ID  EX  MEM WB
```

Here lw uses the memory in cycle 4, so Instr 4 cannot be fetched until cycle 5. **Hence pipelined datapaths need separate instruction and data memories (or separate I- and D-caches).**

3. Worked Example: Data Hazard with No Forwarding (slides 12–15)

```
add $s0, $t0, $t1
sub $t2, $s0, $t3
```

- add **writes** $s0 in **WB (cycle 5)**.
- sub **reads** $s0 in **ID**, normally cycle 3 — too early.

Register-file convention: a register is **written in the first half** of a cycle and **read in the second half** (slide 55). So sub may read $s0 in cycle 5 — the same cycle add writes it.

```
                1   2   3   4   5   6   7   8
add $s0,$t0,$t1 IF  ID  EX  MEM WB
sub $t2,$s0,$t3     IF  ID  st  st  EX  MEM WB
```

2 stall cycles. Without forwarding, the consumer's ID must line up with the producer's WB.

MEMORIZE **No-forwarding rule:** if the consumer is **d** instructions after the producer, it needs **max(0, 3 − d)** stall cycles. Next instruction (d = 1) → **2 stalls**; two later (d = 2) → **1 stall**; three or more later → **0** (the register file handles it).

The same rule applies to loads when there is no forwarding, because the loaded value is also only written in WB.

4. Control Hazard (slide 25)

The pipeline must fetch the next instruction in the cycle after fetching a branch, but the branch outcome is not known yet. MIPS reduces the damage by **comparing registers and computing the target early (in ID)** and by **prediction** — see Lessons 15–16.

Exam Approach

1. List each **RAW dependence**: which later instruction reads a register written by an earlier one.
2. Without forwarding: place each consumer's **ID in the producer's WB cycle** (or later).
3. A stalled instruction also holds back **all instructions behind it** (they wait in IF/ID).
4. Total cycles = cycle in which the last instruction finishes WB = 5 + (n − 1) + total stalls.

TRAP

- An instruction can depend on **more than one** earlier instruction; use the **latest** write of that register.
- The register-file split (write first half, read second half) means **3 apart needs no stall**; don't add an extra cycle.
- A WAR or WAW pair (e.g. a later instruction writes a register an earlier one read) is **not** a hazard in this in-order 5-stage pipeline.

Practice — Lesson 11

Part (a): Actual — 2022-23 (CSE 305) Q1(a). Part (b): Actual — 2016-17 Q4(a) / 2014-15 Q3(a).

(a) Assume pipelined execution of: ins 1: sub $2,$1,$3; ins 2: and $2,$2,$5; ins 3: or $6,$2,$7. **No data forwarding** is available. Construct the execution pipeline with the stalls required for correct output and evaluate the total clock cycles.

(b) What type of pipeline hazard occurs in the following code? How can it be overcome?

```
lw  $t1, 0($t0)
lw  $t2, 4($t0)
add $t3, $t1, $t2
sw  $t3, 12($t0)
lw  $t4, 8($t0)
add $t5, $t1, $t4
sw  $t5, 16($t0)
```

✓ Solution

(a) Dependences: ins 1 → ins 2 ($2) and ins 2 → ins 3 ($2 — the **latest** writer of $2 is ins 2, not ins 1).

```
             1   2   3   4   5   6   7   8   9   10  11
sub $2,$1,$3 IF  ID  EX  MEM WB
and $2,$2,$5     IF  ID  st  st  EX  MEM WB
or  $6,$2,$7         IF  st  st  ID  st  st  EX  MEM WB
```

- and must read $2 in sub's WB cycle (5) → **2 stalls**.
- or must read $2 in and's WB cycle (8) → **2 more stalls** (its two IF holds are only waiting behind and).
- **Total stalls = 4; total cycles = 5 + 2 + 4 = 11.**

(b) Data hazards (RAW). add $t3 needs $t2 loaded by the instruction just before it, and add $t5 needs $t4 loaded just before it — these are **load-use hazards**; sw $t3 needs $t3 from the add just before it. (With one shared memory there would also be structural hazards.)

Overcome by: forwarding (solves the add → sw case with no stall), **one stall cycle per load-use hazard** plus forwarding (13 cycles), or **code scheduling** — move lw $t4 up above the first add so no load is followed by its use (11 cycles, no stalls; see Lesson 14).

Lesson 12 — Forwarding and the Double Data Hazard

Slides: Chapter 4 Part 2, 16 and 53–65

EXAM FOCUS The most repeated pipelining question. **Design a complete forwarding unit with equations, considering double data hazard:** 2023-24 Q1(a), 2021-22 Q2(a), 2017-18 Q2(b), 2015-16 Q3(a), 2012-13 Q4(a). **Write forwarding conditions:** 2019-20 Q8(c), 2022-23 (CSE 305) Q1(c), 2020-21 Q4.

1. Worked Example: Use the Result When It Is Computed (slide 16)

For add $s0,$t0,$t1 followed by sub $t2,$s0,$t3: the sum is **ready at the end of add's EX (cycle 3)** and needed at the **start of sub's EX (cycle 4)**. Send it straight from the pipeline register to the ALU input — **no stall**.

```
                1   2   3   4   5   6
add $s0,$t0,$t1 IF  ID  EX  MEM WB
sub $t2,$s0,$t3     IF  ID  EX  MEM WB
```

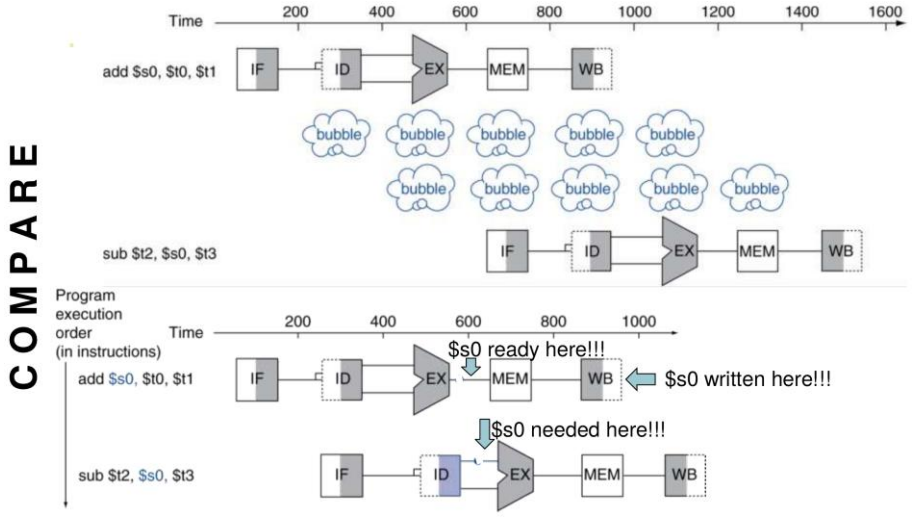


Figure 12.1 — Forwarding (bypassing): $s0 is ready after EX and needed in the next EX (slide 16)

2. A Longer Example (slides 53–55)

Suppose $1 = 30 and $3 = 50, so sub writes $2 = −20.

```
                1   2   3   4   5   6   7   8   9
sub $2,$1,$3    IF  ID  EX  MEM WB
and $12,$2,$5       IF  ID  EX  MEM WB
or  $13,$6,$2           IF  ID  EX  MEM WB
add $14,$2,$2               IF  ID  EX  MEM WB
sw  $15,100($2)                 IF  ID  EX  MEM WB
```

Instruction	Needs $2 in	Gets it from
and $12,$2,$5	EX, cycle 4	**EX/MEM** (sub is in MEM) — forward
or $13,$6,$2	EX, cycle 5	**MEM/WB** (sub is in WB) — forward
add $14,$2,$2	ID, cycle 5	register file (written first half of cycle 5)
sw $15,100($2)	ID, cycle 6	register file

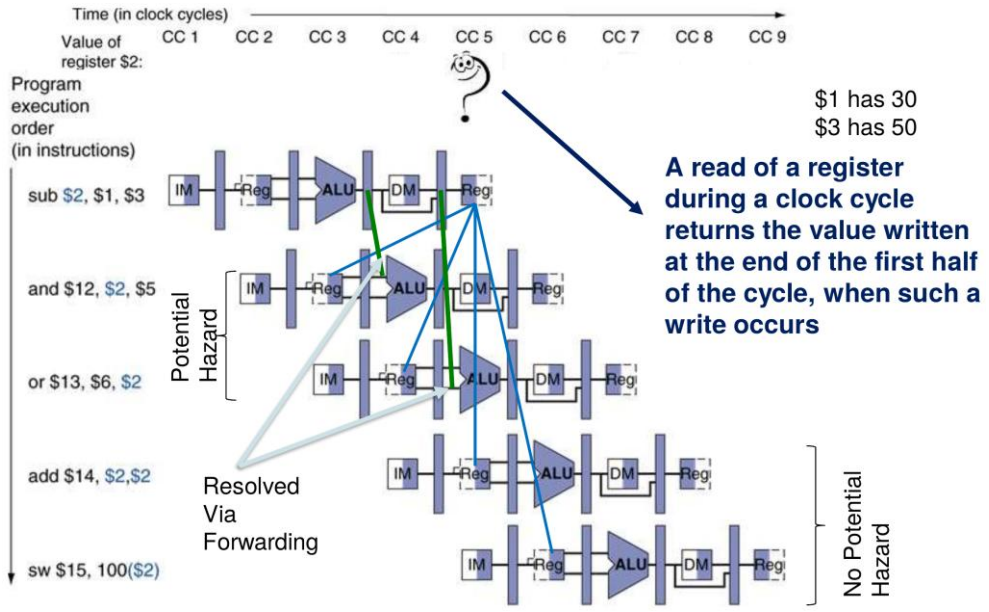


Figure 12.2 — Dependencies resolved by forwarding (slide 55)

3. Detecting the Need to Forward (slides 56–57)

Pass register numbers along the pipeline. The ALU operands of the instruction in EX are **ID/EX.RegisterRs** and **ID/EX.RegisterRt**. A data hazard exists when:

Case	Condition	Forward from
1a	EX/MEM.RegisterRd = ID/EX.RegisterRs	EX/MEM
1b	EX/MEM.RegisterRd = ID/EX.RegisterRt	EX/MEM
2a	MEM/WB.RegisterRd = ID/EX.RegisterRs	MEM/WB
2b	MEM/WB.RegisterRd = ID/EX.RegisterRt	MEM/WB

…but **only if** the forwarding instruction will write a register (**RegWrite = 1**) and its destination is **not $zero** (**RegisterRd ≠ 0**).

> NOTE "RegisterRd" in EX/MEM and MEM/WB means the **destination register number** chosen by the RegDst mux — rd for R-type, but **rt** for lw/addi.

4. The Forwarding Muxes

Each ALU input gets a 3-input mux controlled by the forwarding unit:

Mux control	Source	Meaning
ForwardA = 00	ID/EX	First ALU operand from the register file
ForwardA = 10	EX/MEM	First operand forwarded from the prior ALU result
ForwardA = 01	MEM/WB	First operand forwarded from data memory or an earlier ALU result
ForwardB = 00	ID/EX	Second ALU operand from the register file
ForwardB = 10	EX/MEM	Second operand forwarded from the prior ALU result
ForwardB = 01	MEM/WB	Second operand forwarded from data memory or an earlier ALU result

5. Forwarding Conditions (slide 61)

```
EX hazard
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs))  ForwardA = 10
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRt))  ForwardB = 10

MEM hazard
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs))  ForwardA = 01
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and (MEM/WB.RegisterRd = ID/EX.RegisterRt))  ForwardB = 01
```

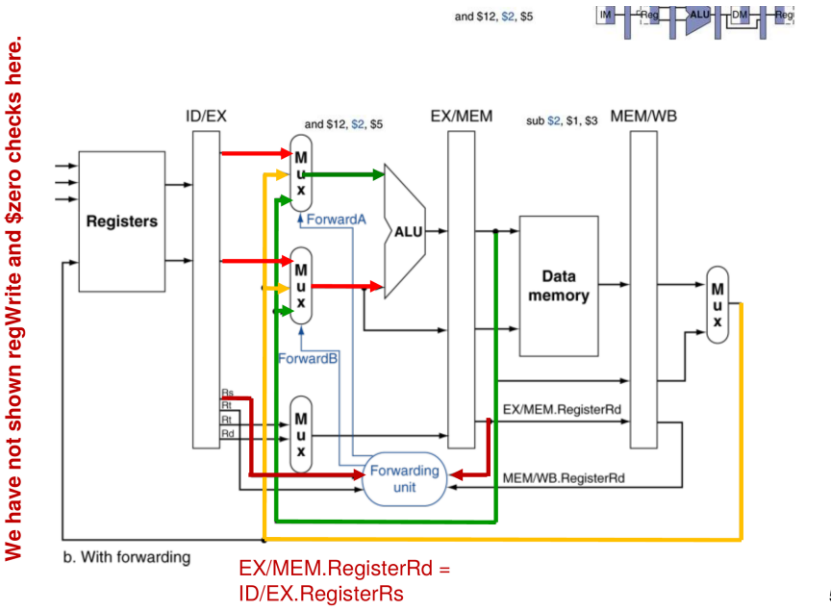


Figure 12.3 — Forwarding paths: EX/MEM → ALU (EX hazard) and MEM/WB → ALU (MEM hazard) (slide 59)

6. Double Data Hazard (slides 62–64)

```
add $1, $1, $2
add $1, $1, $3
add $1, $1, $4
```

Let $1 = 10, $2 = 30, $3 = 20, $4 = 4. The first add gives 40, the second 60, so the third should compute **60 + 4 = 64**.

```
              1   2   3   4   5   6   7
add $1,$1,$2  IF  ID  EX  MEM WB
add $1,$1,$3      IF  ID  EX  MEM WB
add $1,$1,$4          IF  ID  EX  MEM WB
```

When the third add is in EX (cycle 5), **both** conditions are true: EX/MEM holds the second add ($1 = 60) and MEM/WB holds the first add ($1 = 40). The plain MEM-hazard rule would forward **40 — wrong**. We must use the **most recent** value, from EX/MEM.

Revised MEM hazard — forward from MEM/WB only if the EX hazard condition is **not** true:

```
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
             and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs))  ForwardA = 01

if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
             and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRt))  ForwardB = 01
```

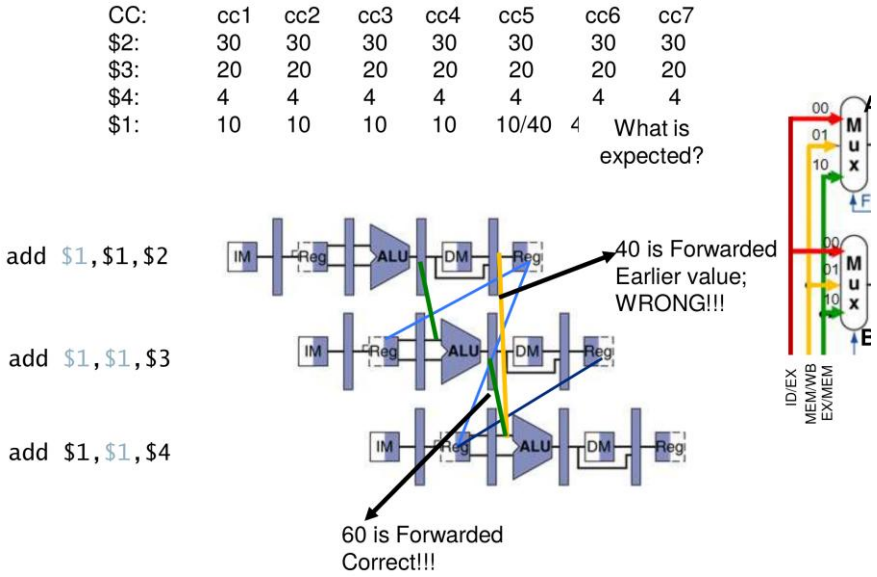


Figure 12.4 — Double data hazard: 40 would be forwarded (wrong), 60 must be forwarded (slide 63)

7. The Forwarding Unit (slide 65)

	Signals
Inputs	ID/EX.RegisterRs, ID/EX.RegisterRt, EX/MEM.RegisterRd, EX/MEM.RegWrite, MEM/WB.RegisterRd, MEM/WB.RegWrite
Outputs	ForwardA (2 bits) → mux at ALU input A; ForwardB (2 bits) → mux at ALU input B
Mux inputs	00: value from ID/EX (register file) · 10: EX/MEM ALU result · 01: MEM/WB write-back value (output of the MemtoReg mux)

```
ID/EX.RegisterRs ──────→ ┌──────────────────────────┐ ──→ ForwardA (2) → ALU-A mux
ID/EX.RegisterRt ──────→ │                          │        00 = ID/EX Read data 1
EX/MEM.RegisterRd ─────→ │    FORWARDING UNIT       │        10 = EX/MEM ALU result
EX/MEM.RegWrite ───────→ │   (equations above,      │        01 = MEM/WB write data
MEM/WB.RegisterRd ─────→ │    revised MEM rule)     │
MEM/WB.RegWrite ───────→ │                          │ ──→ ForwardB (2) → ALU-B mux
                         └──────────────────────────┘        (same codes, Read data 2)
```

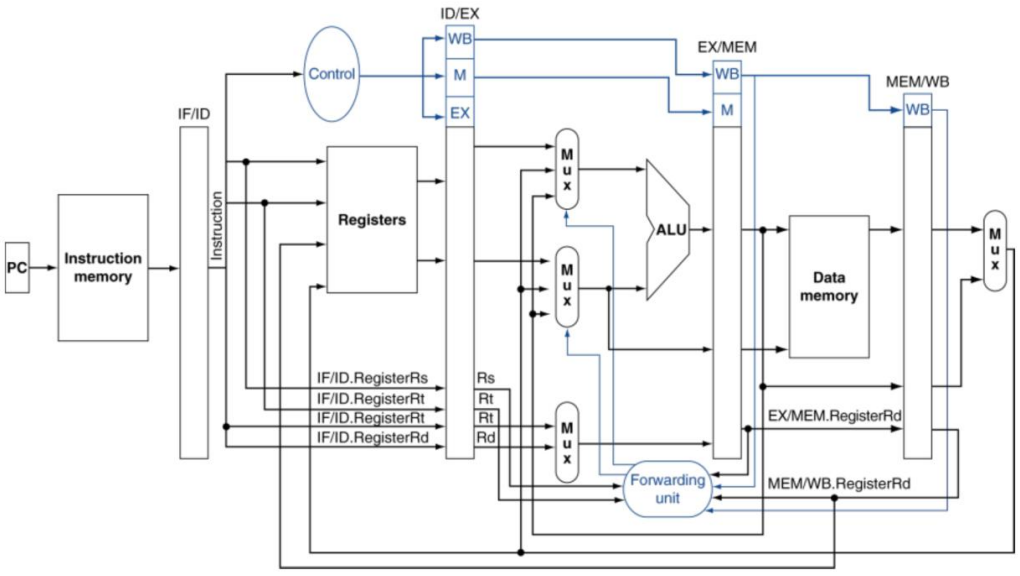


Figure 12.5 — Datapath with forwarding (slide 65)

Exam Approach

"Design a complete forwarding unit considering double data hazard" — write, in this order:

1. Why: an operand needed in EX is still in EX/MEM or MEM/WB.
2. The **inputs/outputs** table and the two 3-input muxes with their 00/10/01 codes.
3. The **EX-hazard** equations (ForwardA/B = 10).
4. The **revised MEM-hazard** equations (ForwardA/B = 01) with the "and not EX hazard" term.
5. A short double-data-hazard example (add $1,$1,$2 ×3) explaining why the EX/MEM value wins.
6. Draw it on the given figure: forwarding unit below the ALU, Rs/Rt from ID/EX, Rd + RegWrite from EX/MEM and MEM/WB, outputs to the two muxes.

TRAP

- Forgetting **RegWrite** (sw and beq do not write registers) or **Rd ≠ 0** ($zero must never be forwarded).
- Swapping the codes: **10 = EX/MEM**, **01 = MEM/WB**.
- Missing the "and not" part — that *is* the double-data-hazard fix.
- Saying forwarding solves **every** hazard: a **load followed by its use** still needs a stall (Lesson 13).

Practice — Lesson 12

Part (a): Actual — 2022-23 (CSE 305) Q1(b),(c). Part (b): Actual — 2019-20 Q8(c). Part (c): Actual — 2020-21 Q4.

(a) For ins 1: sub $2,$1,$3; ins 2: and $2,$2,$5; ins 3: or $6,$2,$7, add data forwarding. Re-evaluate the total clock cycles, explain the double data hazard in this sequence, and write the forwarding condition between ins 1 and ins 2.

(b) For sub $2,$1,$3; and $12,$2,$5; or $13,$6,$2, write the forwarding conditions for the sub → and path and the sub → or path.

(c) In a 5-stage MIPS pipeline, identify all data hazards, mark where forwarding is needed and write the forwarding conditions. Can all of the hazards be solved by forwarding?

```
lw  $t0, 0($t1)
add $t0, $t0, $t1
add $t0, $t2, $t0
add $t0, $t0, $t3
sub $t1, $t2, $t3
and $t4, $t5, $t6
sw  $t0, 0($t6)
```

✓ Solution

(a) With forwarding there are no stalls:

```
             1   2   3   4   5   6   7
sub $2,$1,$3 IF  ID  EX  MEM WB
and $2,$2,$5     IF  ID  EX  MEM WB
or  $6,$2,$7         IF  ID  EX  MEM WB
```

Total = 7 cycles (vs 11 without forwarding).

- **ins 1 → ins 2:** in cycle 4, and is in EX and sub is in EX/MEM → ForwardA = 10.
- **ins 2 → ins 3 (double data hazard):** in cycle 5, or is in EX. **Both** EX/MEM (and, $2) and MEM/WB (sub, $2) have destination $2 = or's Rs. The correct value is the **most recent** one, from and (EX/MEM) → ForwardA = 10; the revised MEM rule blocks the MEM/WB forward.

Forwarding condition between ins 1 and ins 2 (EX hazard):

```
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs))  ForwardA = 10
```

(b) sub → and (green, EX hazard; $2 is and's **Rs**):

```
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs))  ForwardA = 10
```

sub → or (yellow, MEM hazard; $2 is or's **Rt**):

```
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
             and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRt))  ForwardB = 01
```

(c) Dependences (I1…I7 in order):

Pair	Reg	Kind	Resolution
I1 → I2	$t0	**Load-use**	**1 stall**, then MEM/WB → EX (ForwardA = 01)
I2 → I3	$t0	EX hazard (I3's **Rt**)	EX/MEM → EX (ForwardB = 10)
I3 → I4	$t0	EX hazard **and** MEM hazard (I2 also wrote $t0)	**Double data hazard** → take I3's value: ForwardA = 10
I4 → I7	$t0	sw store data, 3 apart	Register file (written first half, read second half) — no forwarding

I5 and I6 use registers no earlier instruction writes (I5 writing $t1 after I2 read it is WAR — not a hazard).

```
                    1   2   3   4   5   6   7   8   9   10  11  12
I1 lw  $t0,0($t1)   IF  ID  EX  MEM WB
I2 add $t0,$t0,$t1      IF  ID  st  EX  MEM WB
I3 add $t0,$t2,$t0          IF  st  ID  EX  MEM WB
I4 add $t0,$t0,$t3              IF  ID  EX  MEM WB
I5 sub $t1,$t2,$t3                  IF  ID  EX  MEM WB
I6 and $t4,$t5,$t6                      IF  ID  EX  MEM WB
I7 sw  $t0,0($t6)                           IF  ID  EX  MEM WB
```

Conditions: I1 → I2: `MEM/WB.RegWrite and MEM/WB.RegisterRd ≠ 0 and MEM/WB.RegisterRd = ID/EX.RegisterRs → ForwardA = 01` (after the stall; the EX/MEM slot holds the bubble). I2 → I3: `EX/MEM.RegWrite and EX/MEM.RegisterRd ≠ 0 and EX/MEM.RegisterRd = ID/EX.RegisterRt → ForwardB = 10`. I3 → I4: `EX/MEM.RegWrite and EX/MEM.RegisterRd ≠ 0 and EX/MEM.RegisterRd = ID/EX.RegisterRs → ForwardA = 10`, with the revised MEM condition preventing ForwardA = 01 from I2.

Can forwarding solve all of them? No. The lw → add pair is a load-use hazard: the data exists only at the end of lw's MEM, after add's EX has started. One stall is unavoidable (12 cycles in total); every other hazard is removed by forwarding.

Lesson 13 — Load-Use Hazard, Stalls and Hazard Detection

Slides: Chapter 4 Part 2, 17–21 and 66–76

EXAM FOCUS **2022-23 (CSE 209) Q2:** identify the hazards, write the detection conditions and **complete the hazard detection unit** with all input/output signals. **2017-18 Q2(a):** how MIPS detects the load hazard and how it stalls. **2012-13 Q4(b):** a case forwarding cannot solve.

1. Worked Example: Forwarding Is Not Enough (slides 17–21)

```
lw  $s0, 20($t1)
sub $t2, $s0, $t3
```

The loaded value exists only at the **end of lw's MEM (cycle 4)**, but sub needs it at the **start of its EX (cycle 4)**. We "can't forward backward in time". Stall **one** cycle, then forward from MEM/WB:

```
                1   2   3   4   5   6   7
lw  $s0,20($t1) IF  ID  EX  MEM WB
sub $t2,$s0,$t3     IF  ID  st  EX  MEM WB
```

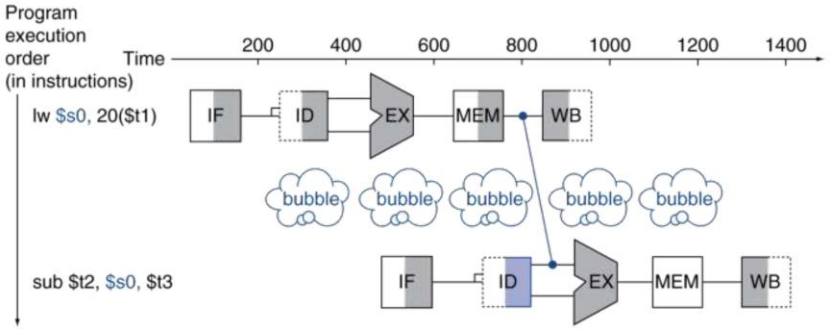


Figure 13.1 — Load-use data hazard: one bubble, then forward (slide 21)

Contrast: load followed by store (slide 66)

```
lw $2, 20($1)
sw $2, 15($3)
```

```
              1   2   3   4   5   6
lw $2,20($1)  IF  ID  EX  MEM WB
sw $2,15($3)      IF  ID  EX  MEM WB
```

The data is ready at the end of lw's MEM (cycle 4) and needed only in **sw's MEM (cycle 5)** — so it *can* be forwarded MEM/WB → data-memory input with an extra path (memory-to-memory copy). By contrast, an ALU instruction that uses the load result needs it a cycle earlier.

2. Detecting the Load-Use Hazard (slides 68–69)

Check when the **using instruction is in ID** and the **load is in EX**:

```
if (ID/EX.MemRead and
    ((ID/EX.RegisterRt = IF/ID.RegisterRs) or
     (ID/EX.RegisterRt = IF/ID.RegisterRt)))
    stall the pipeline (insert a bubble)
```

- **ID/EX.MemRead = 1** identifies a **lw** in EX.
- For lw, **rt is the destination**, so compare ID/EX.RegisterRt with both source fields of the instruction in ID.

3. How to Stall (slides 71–72)

1. **Force the control values in ID/EX to 0** → EX, MEM and WB do a **nop** (the bubble).
2. **Prevent the update of PC and IF/ID** (PCWrite = 0, IF/IDWrite = 0) → the using instruction is **decoded again** and the following instruction is **fetched again**.
3. After the 1-cycle stall, lw is in MEM/WB, so the value is **forwarded to EX**.

```
                 1   2   3   4   5   6   7   8   9   10
lw  $2,20($1)    IF  ID  EX  MEM WB
and $4,$2,$5         IF  ID  st  EX  MEM WB
or  $8,$2,$6             IF  st  ID  EX  MEM WB
add $9,$4,$2                 IF  ID  EX  MEM WB
slt $1,$6,$7                     IF  ID  EX  MEM WB
```

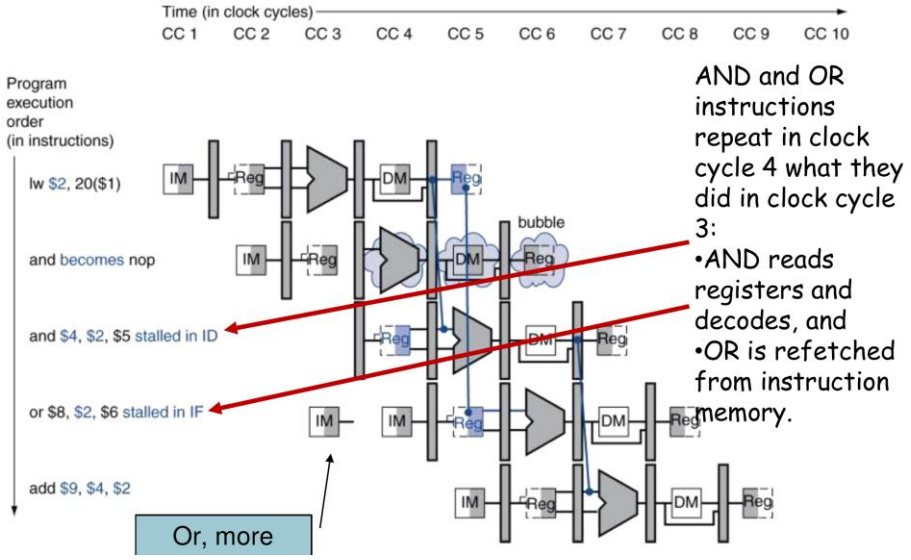


Figure 13.2 — Stall/bubble: and is held in ID, or is refetched (slide 74)

4. The Hazard Detection Unit (slide 75)

	Signals
Inputs	ID/EX.MemRead, ID/EX.RegisterRt, IF/ID.RegisterRs, IF/ID.RegisterRt
Outputs	**PCWrite** (0 = hold PC), **IF/IDWrite** (0 = hold IF/ID), **control-mux select** (1 = pass 0s into the ID/EX control fields)

```
IF/ID.RegisterRs ─┐                    ┌──→ PCWrite ─────→ PC
IF/ID.RegisterRt ─┤    HAZARD          ├──→ IF/IDWrite ──→ IF/ID
ID/EX.RegisterRt ─┤   DETECTION        │
ID/EX.MemRead ────┘     UNIT           └──→ select ──→ MUX: control or 0s
                                                  → ID/EX (WB, M, EX)
```

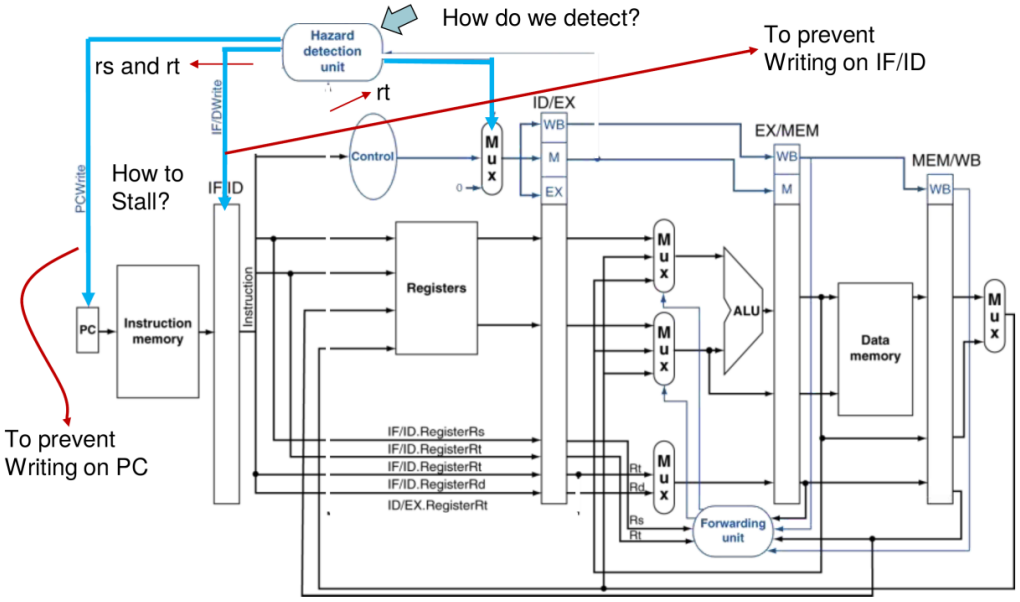


Figure 13.3 — Datapath with hazard detection (slide 75)

MEMORIZE **Detect in ID, stall one cycle, then forward.** Stalls reduce performance but are needed for correct results; the compiler can reorder code to avoid them (slide 76).

Exam Approach

1. For every pair, ask **"is the producer a lw and is the consumer the very next instruction?"** → stall 1 + forward. Otherwise forwarding alone is enough (for ALU-to-ALU pairs).
2. Write the slide-69 condition with the **actual register numbers** for each hazard.
3. When asked to "complete the HDU", draw **all four inputs and three outputs**, with the zeroing mux before the ID/EX control fields.

TRAP

- The stall is **one** cycle with forwarding (two without forwarding).
- Stalling holds **PC and IF/ID**; zeroing goes into **ID/EX** (not EX/MEM).
- The HDU checks **IF/ID** (instruction in ID) against **ID/EX** (load in EX) — the forwarding unit checks ID/EX against EX/MEM and MEM/WB. Do not mix them.

Practice — Lesson 13

Part (a): Actual — 2022-23 (CSE 209) Q2. Part (b): Actual — 2017-18 Q2(a) / 2012-13 Q4(b).

(a) Identify the hazard(s) in the code below. Write the detection condition for each identified hazard and complete the hazard detection unit with all necessary input and output signals.

```
lw   $t0, 55($t1)
addi $t1, $t0, 1
lw   $t0, 57($t1)
sw   $t0, 32($t4)
```

(b) Explain how MIPS detects the hazard due to a load instruction and how it stalls the pipeline. Give a scenario that forwarding cannot solve.

✓ Solution

(a) Three RAW hazards:

Pair	Reg	Kind	Detection condition
lw → addi	$t0	**Load-use** (addi's Rs)	ID/EX.MemRead and (ID/EX.RegisterRt = IF/ID.RegisterRs) → stall 1
addi → lw	$t1	ALU → base address (lw's Rs)	EX/MEM.RegWrite and EX/MEM.RegisterRd ≠ 0 and EX/MEM.RegisterRd = ID/EX.RegisterRs → ForwardA = 10 (no stall)
lw → sw	$t0	Load → store data (sw's **Rt**)	ID/EX.MemRead and (ID/EX.RegisterRt = IF/ID.RegisterRt) → stall 1 with the standard unit

For addi the destination is rt, so "EX/MEM.RegisterRd" here is addi's rt ($t1) as selected by RegDst.

With the standard hazard detection unit (2 stalls, forwarding elsewhere):

```
                     1   2   3   4   5   6   7   8   9   10
I1 lw   $t0,55($t1)  IF  ID  EX  MEM WB
I2 addi $t1,$t0,1        IF  ID  st  EX  MEM WB
I3 lw   $t0,57($t1)          IF  st  ID  EX  MEM WB
I4 sw   $t0,32($t4)              IF  ID  st  EX  MEM WB
```

Total = 10 cycles. (With an extra MEM/WB → memory write-data forwarding path, as on slide 66, the lw → sw stall can be removed, giving 9 cycles.)

Completed HDU: inputs ID/EX.MemRead, ID/EX.RegisterRt, IF/ID.RegisterRs, IF/ID.RegisterRt; outputs PCWrite and IF/IDWrite (to hold PC and IF/ID) and the select of a mux that sends 0s instead of the Control outputs into the WB/M/EX fields of ID/EX — exactly the block in Figure 13.3.

(b) The HDU in ID checks whether the instruction in EX is a load (ID/EX.MemRead) whose destination (ID/EX.RegisterRt) matches a source of the instruction in ID (IF/ID.RegisterRs or IF/ID.RegisterRt). If so it stalls: it zeroes the ID/EX control fields (a bubble), and de-asserts PCWrite and IF/IDWrite so the dependent instruction is decoded again and the next one is fetched again. One cycle later the value is in MEM/WB and is forwarded.

Scenario forwarding cannot solve: lw $s0,20($t1) followed immediately by sub $t2,$s0,$t3 — the data is produced at the end of MEM, after sub's EX would have begun.

Lesson 14 — Timing Diagrams and Code Scheduling

Slides: Chapter 4 Part 2, 22–24 and 76

EXAM FOCUS **The "A = B + C; D = A + E" family** — count cycles, then reschedule: **2022-23 (CSE 209) Q1(c)**, **2021-22 Q1(b)**, **2019-20 Q5(a)–(c)** (A = B + C; D = A + D, with forwarding paths), **2017-18 Q1(a)**, 2015-16 Q2(d). One of the safest marks in the paper if you follow the method.

1. Counting Rules (with Forwarding)

Producer → consumer	Stalls	How it is resolved
ALU instruction → any later use	**0**	Forward from EX/MEM or MEM/WB
lw → **next** instruction uses the value in EX (ALU operand or base address)	**1**	Stall, then forward MEM/WB → EX
lw → an instruction **two or more** later	**0**	Forward MEM/WB, or read the register file
lw → next instruction is a **sw storing that value**	0 or 1	0 with a MEM/WB → MEM path (slide 66); 1 with the standard HDU — **state your assumption**
Branch comparing in ID	see Lesson 15	0, 1 or 2 stalls

Total cycles = 5 + (n − 1) + total stall cycles

2. Worked Example: Slide 22 Code

C code: **A = B + E; C = B + F;** with A(12), B(0), E(4), C(16), F(8) as offsets from $t0.

```
                1   2   3   4   5   6   7   8   9   10  11  12  13
lw  $t1,0($t0)  IF  ID  EX  MEM WB
lw  $t2,4($t0)      IF  ID  EX  MEM WB
add $t3,$t1,$t2         IF  ID  st  EX  MEM WB
sw  $t3,12($t0)             IF  st  ID  EX  MEM WB
lw  $t4,8($t0)                  IF  ID  EX  MEM WB
add $t5,$t1,$t4                 IF  ID  st  EX  MEM WB
sw  $t5,16($t0)                     IF  st  ID  EX  MEM WB
```

- add $t3 uses $t2 loaded just before it → **1 stall**.
- add $t5 uses $t4 loaded just before it → **1 stall**.
- sw $t3 / sw $t5 get their data from the add just before them by forwarding — no stall.

7 instructions, 2 stalls → 5 + 6 + 2 = 13 cycles.

Reschedule: move the third load up

lw $t4 does not depend on anything above it, and its address (8) differs from the store addresses (12, 16), so it can move up:

```
                1   2   3   4   5   6   7   8   9   10  11
lw  $t1,0($t0)  IF  ID  EX  MEM WB
lw  $t2,4($t0)      IF  ID  EX  MEM WB
lw  $t4,8($t0)          IF  ID  EX  MEM WB
add $t3,$t1,$t2             IF  ID  EX  MEM WB
sw  $t3,12($t0)                 IF  ID  EX  MEM WB
add $t5,$t1,$t4                 IF  ID  EX  MEM WB
sw  $t5,16($t0)                     IF  ID  EX  MEM WB
```

Now every load is at least one instruction away from its use: **0 stalls, 11 cycles**.

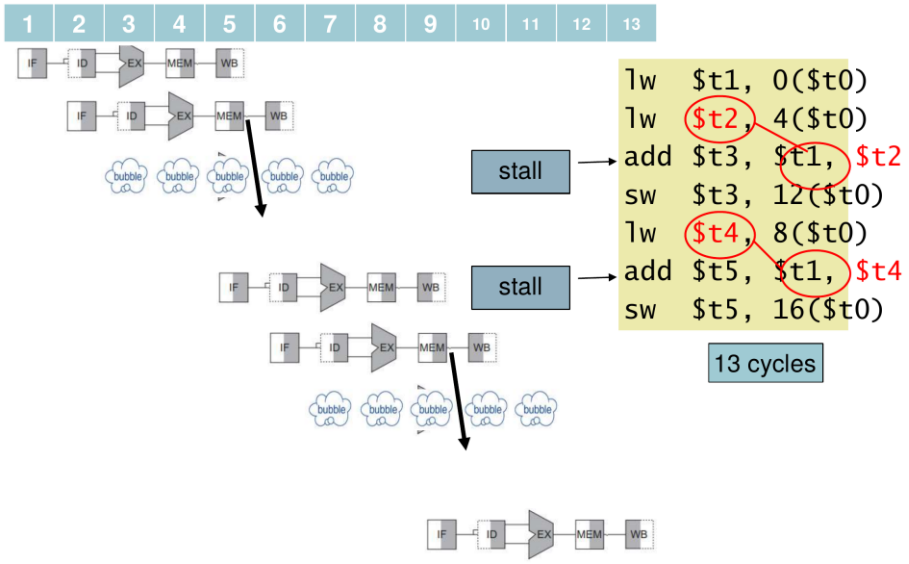


Figure 14.1 — Original order: 2 stalls, 13 cycles (slide 23)

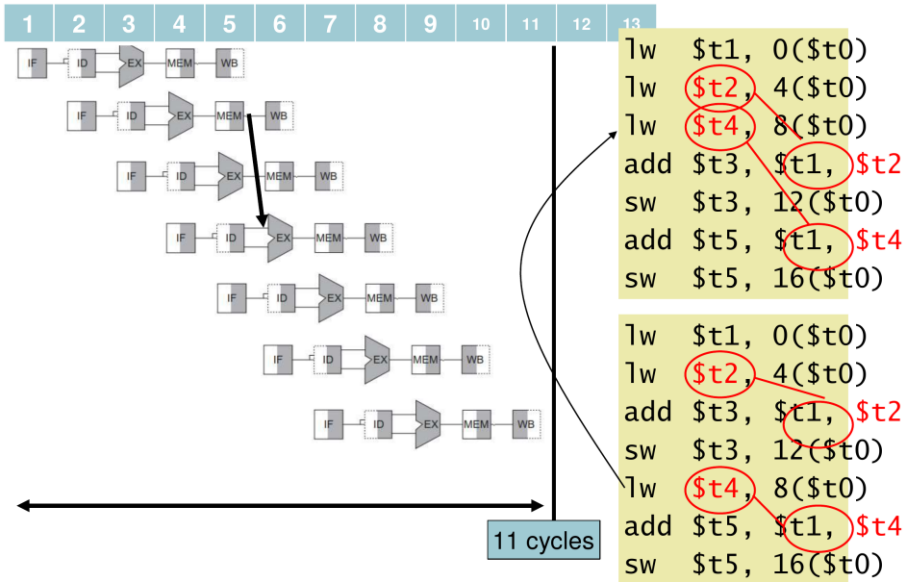


Figure 14.2 — Scheduled order: 11 cycles (slide 24)

MEMORIZE **Scheduling recipe:** (1) find each lw immediately followed by its use; (2) find an **independent** instruction (usually a later load) and move it into the gap; (3) never move an instruction above the one that produces its operand, and never move a load above a store to the **same address**.

Exam Approach

1. Translate the C code: one lw per source variable, the arithmetic, one sw per assigned variable. Write the offsets as comments.
2. Mark each RAW dependence and classify it with the table above.
3. Draw the multi-cycle diagram (IF ID EX MEM WB staircase) with **st** for each stall and arrows (or notes) for forwarding.
4. Count: **5 + (n − 1) + stalls**.
5. Reschedule by hoisting independent loads; recount; say **why** it is legal.

TRAP

- Loads that are **two** instructions before the use cause **no** stall with forwarding.
- If the question says **no forwarding**, use the Lesson 11 rule (2 stalls for adjacent dependences) instead.
- Code reordering is an application of **"performance via pipelining"** among the eight great ideas (2020-21 Q1(a)): it removes pipeline stalls.

Practice — Lesson 14

Part (a): Actual — 2022-23 (CSE 209) Q1(c) and 2017-18 Q1(a). Part (b): Actual — 2019-20 Q5(a)–(c).

(a) A = B + C; D = A + E; — all are memory variables. The base address is in $t0 and the offsets of A, B, C, D, E are 0, 4, 8, 12, 16. Write the MIPS code and find the number of cycles. Can code scheduling reduce it? If yes, rearrange and recount.

(b) The code below computes A = B + C; D = A + D. Draw the execution pipeline showing stalls and forwarding paths (forwarding is available). How many stalls and cycles are required? Can the code be rearranged to reduce the stalls?

```
lw  $t1, 0($t0)      // load B
lw  $t2, 4($t0)      // load C
add $t3, $t1, $t2    // A = B + C
sw  $t3, 12($t0)     // store A
lw  $t4, 8($t0)      // load D
add $t4, $t3, $t4    // D = A + D
sw  $t4, 8($t0)      // store D
```

✓ Solution

(a) Original code (2 load-use stalls):

```
                    1   2   3   4   5   6   7   8   9   10  11  12  13
lw  $t1,4($t0)  #B  IF  ID  EX  MEM WB
lw  $t2,8($t0)  #C      IF  ID  EX  MEM WB
add $t3,$t1,$t2 #A          IF  ID  st  EX  MEM WB
sw  $t3,0($t0)              IF  st  ID  EX  MEM WB
lw  $t4,16($t0) #E              IF  ID  EX  MEM WB
add $t5,$t3,$t4 #D                  IF  ID  st  EX  MEM WB
sw  $t5,12($t0)                         IF  st  ID  EX  MEM WB
```

13 cycles. Hoist the load of E (address 16 is never stored to here):

```
                    1   2   3   4   5   6   7   8   9   10  11
lw  $t1,4($t0)  #B  IF  ID  EX  MEM WB
lw  $t2,8($t0)  #C      IF  ID  EX  MEM WB
lw  $t4,16($t0) #E          IF  ID  EX  MEM WB
add $t3,$t1,$t2 #A              IF  ID  EX  MEM WB
sw  $t3,0($t0)                  IF  ID  EX  MEM WB
add $t5,$t3,$t4 #D                  IF  ID  EX  MEM WB
sw  $t5,12($t0)                         IF  ID  EX  MEM WB
```

0 stalls → 11 cycles.

(b) Same shape as the slide example: **2 stalls, 13 cycles**.

Consumer	Needs	Path
add $t3,$t1,$t2	$t2 from the lw just before	**1 stall**, then MEM/WB → EX (ForwardB = 01); $t1 read from the register file
sw $t3,12($t0)	$t3 (store data)	EX/MEM → EX (ForwardB = 10)
add $t4,$t3,$t4	$t4 from the lw just before	**1 stall**, then MEM/WB → EX (ForwardB = 01); $t3 from the register file
sw $t4,8($t0)	$t4 (store data)	EX/MEM → EX (ForwardB = 10)

Rearranged: lw $t1; lw $t2; **lw $t4, 8($t0)**; add $t3,$t1,$t2; sw $t3,12($t0); add $t4,$t3,$t4; sw $t4,8($t0). Moving the load of D above the store of A is legal because the addresses differ (8 vs 12). **0 stalls, 11 cycles.** Forwarding then: lw $t2 → add $t3 via MEM/WB; add $t3 → sw via EX/MEM; add $t3 → add $t4 via MEM/WB; add $t4 → sw via EX/MEM.

Lesson 15 — Branch Hazards

Slides: Chapter 4 Part 2, 25–31 and 77–81

EXAM FOCUS **2023-24 Q1(b)** — count stalls/cycles for code with two branches (branch in ID, forwarding, taken penalty 1, load-use 1). **Data hazards for branches** (0/1/2 stalls): 2021-22 Q1(c), 2017-18 Q2(c), 2013-14 Q8(c). **Flush and stall together:** 2022-23 Q5, 2018-19 Q2(a). **Jump stall:** 2022-23 Q7. **ID-stage branch hardware:** 2022-23 (CSE 305) Q4(b).

1. Worked Example: Where Is the Branch Decided?

```
36: sub $10, $4, $8
40: beq $1, $3, 7        # taken: target = 44 + 7 × 4 = 72
44: and $12, $2, $5
48: or  $13, $2, $6
52: add $14, $4, $2
56: slt $15, $6, $7
 ...
72: lw  $4, 50($7)
```

Decided in MEM (basic pipeline, slide 77)

While beq travels to MEM, three more instructions (44, 48, 52) are fetched. When the branch is taken they must be **flushed** (control values set to 0):

```
                  1   2   3   4   5   6   7   8   9
40 beq $1,$3,7    IF  ID  EX  MEM WB
44 and $12,$2,$5      IF  ID  EX  x
48 or  $13,$6,$2          IF  ID  x
52 add $14,$2,$2              IF  x
72 lw  $4,50($7)                  IF  ID  EX  MEM WB
```

3-cycle penalty for every taken branch.

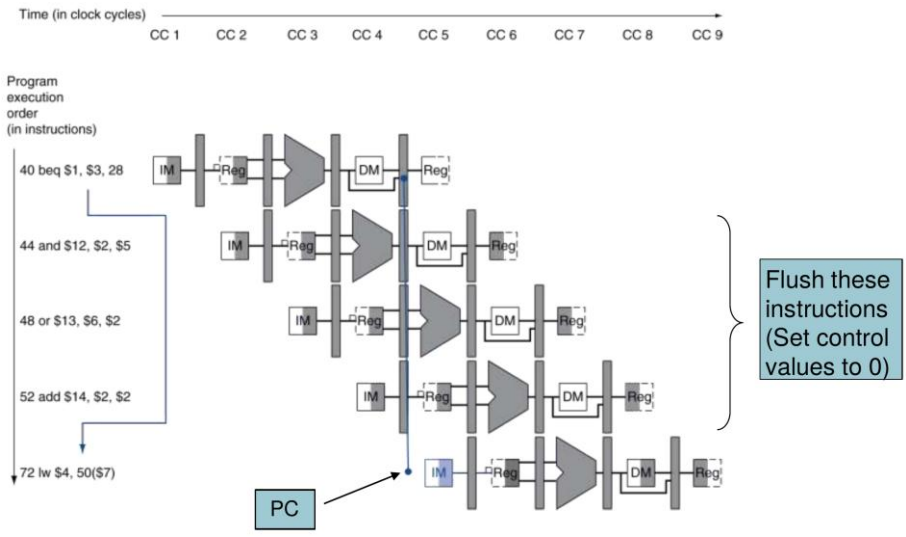


Figure 15.1 — Branch decided in MEM: flush three instructions (slide 77)

Decided in ID (slide 78)

Move the **target-address adder** and a **register comparator** into ID. Only the one instruction fetched behind the branch must be flushed:

```
                  1   2   3   4   5   6   7
40 beq $1,$3,7    IF  ID  EX  MEM WB
44 and $12,$2,$5      IF  x
72 lw  $4,50($7)          IF  ID  EX  MEM WB
```

1-cycle penalty for a taken branch. MIPS uses this design.

2. Three Ways to Deal with the Remaining Cycle

Method	Idea
Stall on branch (slide 27)	Always wait until the branch is decided in ID → 1 bubble per branch
Predict not taken (slides 28–29)	Fetch PC + 4 anyway; if the branch is taken, flush that one instruction (IF.Flush zeroes IF/ID) — no penalty when the prediction is right
Delayed branch (slide 31)	Always execute the next sequential instruction; the compiler fills that "delay slot" with useful work

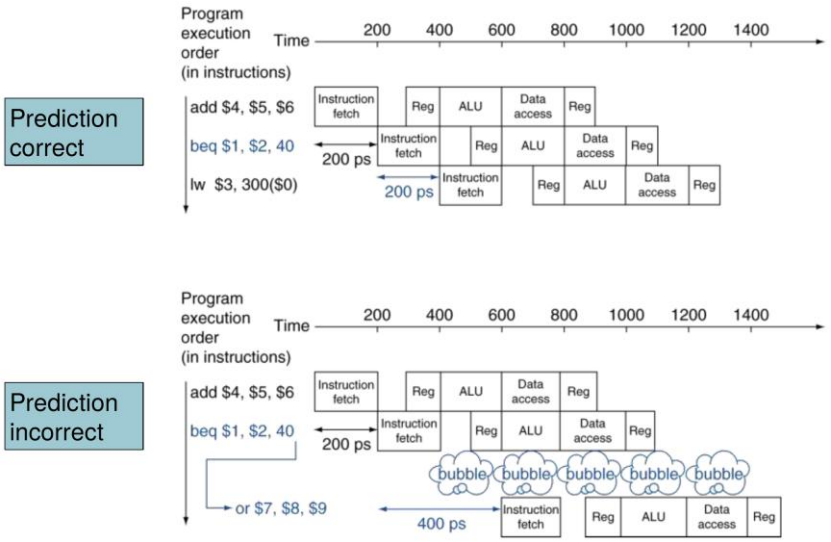


Figure 15.2 — Predict not taken: correct (top) and incorrect (bottom) (slide 29)

3. The Four ID-Stage Additions (2022-23 CSE 305 Q4(b))

To decide beq by the end of ID:

1. **Shift-left-2** unit for the sign-extended offset.
2. **Branch-target adder** (PC + 4 from IF/ID + shifted offset) — moved from EX to ID.
3. **Register equality comparator** (XOR the two Read-data values bit by bit, then OR/NOR → equal?) — replaces using the ALU's Zero in EX.
4. **IF.Flush** control (AND of Branch and "equal") that **zeroes IF/ID** and selects the target in the PC mux.

How it works by the end of ID: the registers are read early in ID and compared; at the same time the offset is sign-extended, shifted and added to PC + 4. If Branch AND equal, PCSrc selects the target (PC ← target at the clock edge) and IF.Flush turns the wrongly fetched instruction into a nop. (Forwarding and hazard detection must also be extended to the comparator — next section.)

4. Data Hazards for Branches (slides 79–81)

Because the comparison now happens in **ID**, the branch needs its registers **earlier** than an ALU instruction would:

Comparison register written by…	Stalls	Example
2nd or 3rd **preceding ALU** instruction	**0**	forward to the comparator
Immediately preceding ALU instruction, or **2nd preceding load**	**1**	value ready only at end of EX / MEM
Immediately preceding load	**2**	value ready only at end of MEM

```
                  1   2   3   4   5   6   7   8
add $1,$2,$3      IF  ID  EX  MEM WB
add $4,$5,$6          IF  ID  EX  MEM WB
add $7,$8,$9              IF  ID  EX  MEM WB
beq $1,$4,target              IF  ID  EX  MEM WB
```

```
                  1   2   3   4   5   6   7
add $1,$2,$3      IF  ID  EX  MEM WB
beq $1,$4,target      IF  ID  st  EX  MEM WB
```

```
                  1   2   3   4   5   6   7   8
lw  $1,addr       IF  ID  EX  MEM WB
add $4,$5,$6          IF  ID  EX  MEM WB
beq $1,$4,target          IF  ID  st  EX  MEM WB
```

```
                  1   2   3   4   5   6   7   8
lw  $1,addr       IF  ID  EX  MEM WB
beq $1,$0,target      IF  ID  st  st  EX  MEM WB
```

5. Can Flush and Stall Happen Together? (2022-23 Q5, 2018-19 Q2(a))

```
beq $1, $2, TARGET   # taken; decided with the ALU Zero (in MEM)
lw  $3, 40($4)
add $2, $3, $4
sw  $2, 40($4)
TARGET: or $1, $1, $2
```

```
                    1   2   3   4   5   6   7   8   9
beq $1,$2,TARGET    IF  ID  EX  MEM WB
lw  $3,40($4)           IF  ID  EX  x
add $2,$3,$4                IF  ID  x
sw  $2,40($4)                   IF  x
TARGET: or $1,$1,$2                 IF  ID  EX  MEM WB
```

Yes. In **cycle 4**: beq is in MEM and is taken → **flush** IF, ID and EX and load the target into PC. At the same moment lw $3 is in EX and add (which reads $3) is in ID → the HDU sees a **load-use hazard** and requests a **stall**: PCWrite = 0, IF/IDWrite = 0, bubble into ID/EX.

The issue: the stall blocks the PC update, so the branch target is **not loaded** (the PC keeps the wrong-path address), and IF/ID is held instead of flushed. The taken branch would be lost and wrong-path instructions would continue.

Solution: give the **flush priority**. The stall is pointless because the instructions behind a taken branch are being discarded anyway:

$$\text{stall} = (\text{load-use condition}) \text{ AND NOT}(\text{PCSrc})$$
$$\text{PCWrite} = \text{IF/IDWrite} = \text{NOT}(\text{stall})$$

So when the branch is taken, PC loads the target and IF/ID and ID/EX are flushed; the stall request is ignored.

6. Jumps in the Pipeline (2022-23 Q7)

- **Chandler: "every jump needs a 1-cycle stall" — agree** for the basic pipeline. j is recognised only when decoded in **ID**; by then the instruction at PC + 4 is already in IF and must be flushed.
- **Joey: "move the control unit to IF" — problem:** decoding can start only after the instruction-memory read finishes, so IF would contain *memory access + decode + jump-address formation* in series. IF becomes the longest stage and the **clock period grows for every instruction**, to save one cycle on jumps only ("make the common case fast" is violated).
- **Ross's solution — a branch/jump target buffer:** a small cache indexed by the PC **during IF**. If the fetched PC hits in the buffer as a jump, its target is supplied immediately and the next fetch is from

the target — **no stall**. Only the first time a jump is seen (buffer miss) does the 1-cycle penalty occur, and the clock period is unchanged. (A delay slot filled by the compiler is the other classic fix.)

Exam Approach — Cycle Counting with Branches

1. State the assumptions (where branches resolve, forwarding, penalties).
2. Data stalls: load-use = 1; for a branch in ID, use the table in §4.
3. Control stalls: add the **taken-branch penalty** only for branches that are taken; with predict-not-taken, a not-taken branch costs 0.
4. Follow the **actual path** of execution for each case — skipped instructions are not counted.
5. Cycles = 5 + (executed instructions − 1) + all stalls.

TRAP

- A branch in ID needs **1 stall** after an ALU instruction that writes its register — an ALU-to-ALU pair would need none.
- The branch penalty is about **control**; branch data stalls are **extra**.
- When a branch is taken, instructions on the skipped path add **nothing** (they are flushed).

Practice — Lesson 15

Part (a): Actual — 2023-24 Q1(b). Part (b): Actual — 2018-19 Q2(b). Part (c): Actual — 2021-22 Q1(c).

(a) Assume: 5-stage MIPS (IF–ID–EX–MEM–WB); code rescheduling not available; all-stage forwarding available; branch resolved in ID; taken-branch penalty = 1 cycle; load-use hazard = 1 stall. Identify all data and control hazards and evaluate: (i) total stall cycles; (ii) total cycles if both branches are not taken; (iii) total cycles if the first branch is taken; (iv) total cycles if the first branch is not taken but the second is taken.

```
lw  $t0, 0($s0)
add $t1, $t0, $s1
beq $t1, $zero, L1
add $t2, $t1, $s2
bne $t2, $zero, L2
sub $t3, $t2, $t1
L1: and $t4, $t1, $t2
L2: or  $t5, $t3, $t4
```

(b) Assume extra hardware decides the branch as early as possible (ID). Discuss the issues in the code below and find the number of cycles.

```
lw  $1, 100($2)
add $1, $1, $2
beq $1, $2, TARGET
```

(c) Give MIPS code examples of data hazards for branches that (i) need no stall, (ii) need one stall, (iii) need two stalls to resolve.

✓ Solution

(a) Hazards:

Pair	Kind	Stalls
lw → add $t1 ($t0)	Load-use	**1**
add $t1 → beq ($t1)	Branch in ID uses the result of the **immediately preceding ALU** instruction	**1**
add $t2 → bne ($t2)	Same	**1**
add $t2 → sub, sub → or, and → or	ALU → ALU	0 (forwarding)
beq, bne	**Control hazards**	+1 only if taken

(ii) Both not taken — all 8 instructions, 3 data stalls:

```
                    1   2   3   4   5   6   7   8   9   10  11  12  13  14  15
lw  $t0,0($s0)      IF  ID  EX  MEM WB
add $t1,$t0,$s1         IF  ID  st  EX  MEM WB
beq $t1,$zero,L1            IF  st  ID  st  EX  MEM WB
add $t2,$t1,$s2                 IF  st  ID  EX  MEM WB
bne $t2,$zero,L2                    IF  ID  st  EX  MEM WB
sub $t3,$t2,$t1                         IF  st  ID  EX  MEM WB
L1: and $t4,$t1,$t2                         IF  ID  EX  MEM WB
L2: or  $t5,$t3,$t4                             IF  ID  EX  MEM WB
```

Cycles = 5 + 7 + 3 = **15**

(iii) First branch taken — executes lw, add, beq, and, or (5 instructions); stalls = 1 (load-use) + 1 (beq data) + 1 (taken penalty) = 3:

```
                    1   2   3   4   5   6   7   8   9   10  11  12
lw  $t0,0($s0)      IF  ID  EX  MEM WB
add $t1,$t0,$s1         IF  ID  st  EX  MEM WB
beq $t1,$zero,L1            IF  st  ID  st  EX  MEM WB
add $t2,$t1,$s2                 IF  st  x
L1: and $t4,$t1,$t2                 IF  ID  EX  MEM WB
L2: or  $t5,$t3,$t4                     IF  ID  EX  MEM WB
```

Cycles = 5 + 4 + 3 = **12**

(iv) First not taken, second taken — executes lw, add, beq, add, bne, or (6 instructions); stalls = 1 + 1 + 1 (data) + 1 (bne taken) = 4:

```
                    1   2   3   4   5   6   7   8   9   10  11  12  13  14
lw  $t0,0($s0)      IF  ID  EX  MEM WB
add $t1,$t0,$s1         IF  ID  st  EX  MEM WB
beq $t1,$zero,L1            IF  st  ID  st  EX  MEM WB
add $t2,$t1,$s2                 IF  st  ID  EX  MEM WB
bne $t2,$zero,L2                    IF  ID  st  EX  MEM WB
sub $t3,$t2,$t1                         IF  st  x
L2: or  $t5,$t3,$t4                             IF  ID  EX  MEM WB
```

Cycles = 5 + 5 + 4 = **14**

(i) Total stall cycles: data-hazard stalls = **3** whenever both branches execute (cases ii, iv) — 2 in case (iii) because bne is skipped — plus 1 per taken branch: case (ii) **3**, case (iii) **3**, case (iv) **4**.

(b) Issues: lw → add is a **load-use** hazard (1 stall, then forward); add → beq: the branch compares in ID using the result of the **immediately preceding ALU** instruction (1 stall, then forward EX/MEM → comparator). The branch itself is a control hazard (1-cycle penalty if taken, not counted here).

```
                  1   2   3   4   5   6   7   8   9
lw  $1,100($2)    IF  ID  EX  MEM WB
add $1,$1,$2          IF  ID  st  EX  MEM WB
beq $1,$2,TARGET          IF  st  ID  st  EX  MEM WB
```

9 cycles (3 instructions + 4 + 2 stalls).

(c) (i) **No stall** — comparison registers written by the 2nd/3rd preceding ALU instructions: add $1,$2,$3; add $4,$5,$6; (another instruction); beq $1,$4,target. (ii) **One stall** — written by the immediately preceding ALU instruction (add $1,$2,$3; beq $1,$4,target), or by the 2nd preceding load (lw $1,addr; add $4,$5,$6; beq $1,$4,target). (iii) **Two stalls** — written by the immediately preceding load: lw $1,addr; beq $1,$0,target. (See the four diagrams in §4.)

Lesson 16 — Branch Prediction

Slides: Chapter 4 Part 2, 28–31 and 82–85

EXAM FOCUS **1-bit predictor on nested loops:** 2023-24 Q1(c), 2021-22 Q1(d), 2017-18 Q2(d), 2013-14 Q8(d). **"Even with a perfect branch predictor, 1-cycle penalty for a taken branch":** 2023-24 Q1(c), 2017-18 Q1(d), 2015-16 Q2(e). **2-bit FSM:** 2019-20 Q8(b). **2-bit accuracy trace:** older paper (TF p. 85). **Dynamic prediction techniques:** 2014-15 Q3(b).

1. Static vs Dynamic Prediction (slides 28, 30)

Longer pipelines cannot decide branches early, so the stall penalty becomes unacceptable. **Predict** the outcome and stall only when the prediction is wrong.

- **Static prediction** (fixed rule, based on typical behaviour): **backward branches taken** (loops), **forward branches not taken** (if-statements). MIPS's simplest form: predict not taken.
- **Dynamic prediction** (hardware measures actual behaviour): record the recent history of each branch and assume the trend continues. When wrong: stall while re-fetching and update the history.

Branch prediction buffer / branch history table (slide 82): a small table **indexed by the (low-order bits of the) branch address**, storing taken / not taken. To execute a branch: check the table, expect the same outcome, start fetching from the fall-through or the target; **if wrong, flush the pipeline and flip the prediction**.

2. Worked Example: Why a 1-Bit Predictor Fails on Nested Loops (slide 83)

```
outer:  …
inner:  …
        beq $t0, $zero, inner     # taken while the inner loop repeats
        …
        beq $t1, $zero, outer
```

Let the inner loop run **10 times** per outer iteration: its branch is **taken 9 times, then not taken once**.

Inner branch	1-bit prediction (= last outcome)	Result
1st execution of this round	Not taken (last time it exited)	**Wrong**
Executions 2–9	Taken	Correct
10th (loop exit)	Taken	**Wrong**

Two mispredictions per inner loop → 8/10 = 80 %, even though the branch is taken 90 % of the time.

Fix — 2-bit predictor (slide 84): change the prediction only after **two successive mispredictions**. The single exit only moves it from "strongly taken" to "weakly taken", so the next round's first branch is still predicted taken → **1 misprediction → 9/10 = 90 %**.

3. The 2-Bit Predictor FSM

Four states — **strongly taken, weakly taken, weakly not taken, strongly not taken**. A correct prediction moves toward the strong state of that side; a misprediction moves one step toward the other side, so the prediction flips only after **two mispredictions in a row**.

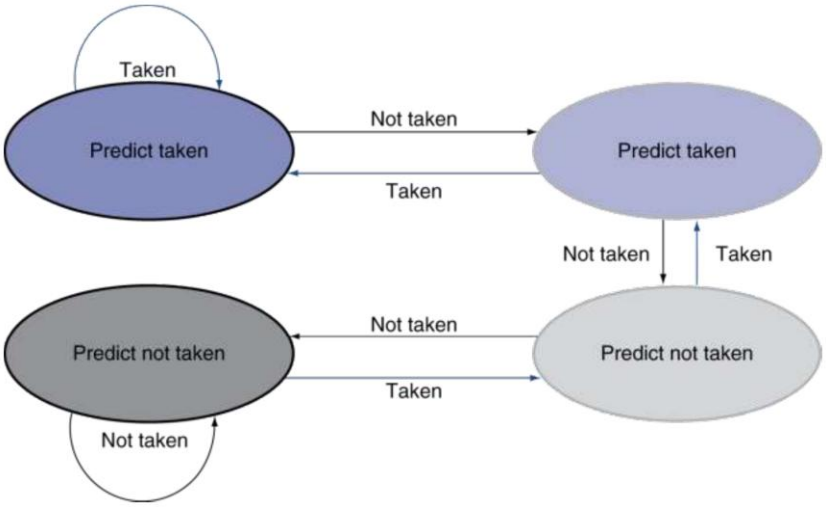


Figure 16.1 — 2-bit predictor: only change prediction on two successive mispredictions (slide 84)

4. "Even with a Perfect Predictor, 1-Cycle Penalty for a Taken Branch" (slide 85)

The predictor says *taken*, but the **target address** is calculated only in **ID**. During the branch's IF cycle the hardware does not yet know it is a branch or where it goes, so the next fetch is from PC + 4 → **one lost cycle**.

Solution — Branch Target Buffer (BTB): a cache of **target addresses**, indexed by the **PC when the instruction is fetched**. If the PC hits and the branch is predicted taken, the target is fetched **in the very next cycle** — no penalty. (A delay slot filled by the compiler is the other option.)

5. Delayed Branch (slide 31)

Always execute the **next sequential instruction** after a branch (the delay slot); the taken branch takes effect after it. The compiler moves an independent instruction into the slot, hiding the delay. This works for a 1-cycle delay; with deeper pipelines and multiple issue, dynamic prediction is better.

Exam Approach

- **Nested loop question:** name the two mispredictions (last inner iteration predicted taken; first iteration of the next round predicted not taken) → fix with a 2-bit predictor; give accuracies if numbers are given.
- **Perfect predictor question:** "prediction ≠ target" → BTB indexed by PC in IF.
- **Accuracy trace:** write a table (state, prediction, actual, correct?, next state); continue until the pattern repeats for the steady state.

TRAP

- A 2-bit predictor does **not** eliminate the loop-exit misprediction — it only removes the second one.
- Start from the state the question gives (e.g. "bottom-left = strongly not taken").
- A predictor without a BTB still pays the taken-branch cycle.

Practice — Lesson 16

Part (a): Actual — 2023-24 Q1(c). Part (b): Actual — older paper (TF p. 85) Q4(e). Part (c): Actual — 2019-20 Q8(b).

(a) For the nested loop above, explain the problem if we use a 1-bit branch predictor and how it can be solved. "Even with a perfect branch predictor, 1-cycle penalty for a taken branch" — why does this penalty occur, and how can it be eliminated?

(b) A branch repeats the outcome pattern **T, T, NT, T**. What is the accuracy of the 2-bit predictor for this sequence, and in the steady state if it repeats forever? The predictor starts in the bottom-left state (strongly predict not taken).

(c) Draw the finite state machine of a 2-bit dynamic branch prediction scheme.

✓ Solution

(a) With a 1-bit predictor the inner-loop branch is mispredicted **twice** per inner loop: on the **last** iteration it is predicted taken (it was taken last time) but falls through; on the **first** iteration of the next outer pass it is predicted not taken (last outcome) but is taken. Solution: a **2-bit predictor**, which changes its prediction only after two consecutive mispredictions, so only the exit is mispredicted.

Even a perfect predictor only says *taken*; the **target** is computed in ID, so the instruction at PC + 4 has already been fetched → 1 cycle lost. Eliminate it with a **Branch Target Buffer**, a cache of targets indexed by the PC in IF: on a hit for a predicted-taken branch the target is fetched immediately.

(b) States: SNT (strong not taken), WNT, WT, ST.

Pass	Outcome	State before	Predicts	Correct?	State after
1	T	SNT	NT	✗	WNT
1	T	WNT	NT	✗	WT
1	NT	WT	T	✗	WNT
1	T	WNT	NT	✗	WT
2	T	WT	T	✓	ST
2	T	ST	T	✓	ST
2	NT	ST	T	✗	WT
2	T	WT	T	✓	ST
3+	T, T, NT, T	ST	T, T, T, T	✓ ✓ ✗ ✓	ST

First pass: 0/4 = 0 %. Steady state: 3/4 = 75 %. (For comparison, a 1-bit predictor reaches only 50 % on this pattern — it mispredicts both the NT and the T that follows it.)

(c) Four states; two "predict taken" and two "predict not taken". A correct prediction moves toward the strong state; a misprediction moves one step toward the other side, so the prediction flips only after **two** mispredictions in a row — the FSM in §3 / Figure 16.1.

Lesson 17 — Multiple Issue, Scheduling and Loop Unrolling

Slides: Chapter 4 Part 2, 86–99

EXAM FOCUS **Dual-issue scheduling of the array loop, IPC, and a technique to increase IPC:** 2023-24 Q4(b), 2021-22 Q2(c). **Refactor a loop to maximise IPC:** 2020-21 Q5. **Block diagram of a dynamically scheduled multiple-issue superscalar CPU:** 2021-22 Q2(b). Older: superscalar vs VLIW (2014-15).

1. Instruction-Level Parallelism (slide 86)

Pipelining already executes several instructions in parallel. To get more ILP:

- **Deeper pipeline** — less work per stage → shorter clock cycle.
- **Multiple issue** — replicate pipeline stages (several pipelines) and **start several instructions per clock cycle**. Then CPI < 1, so we use **IPC (instructions per cycle)**.

Example: a **4 GHz, 4-way** multiple-issue processor peaks at **16 BIPS**, peak **CPI = 0.25**, peak **IPC = 4** — dependencies reduce this in practice.

	Static multiple issue (VLIW)	Dynamic multiple issue (superscalar)
Who groups instructions	**Compiler**, into "issue packets / slots"	**CPU** examines the instruction stream each cycle and issues 0, 1, 2, …
Hazards	Compiler detects and avoids them (pads with nop)	CPU resolves them at run time; compiler scheduling still helps
Idea	An issue packet = one **very long instruction word**	Code semantics ensured by the CPU

2. Speculation (slides 88–89)

Guess what to do with an instruction, start it as soon as possible, then check: if right, complete it; if wrong, **roll back** and do the right thing. Used by both static and dynamic multiple issue.

- Speculate on a **branch outcome** → roll back if the other path is taken.
- Speculate on a **load** (e.g. move it above a store) → roll back if that location was updated.
- **Compiler** speculation: reorder (e.g. move a load before a branch) and add "fix-up" code. **Hardware** speculation: look ahead, **buffer results** until they are known to be needed, flush the buffers on a wrong guess.

3. MIPS with Static Dual Issue (slides 92–94)

- Two-issue packets, **64-bit aligned**: **one ALU/branch instruction, then one load/store instruction**. An unused slot is padded with **nop**.
- The datapath adds a second (address) adder/ALU and more register-file ports so both instructions run together.

Address	Type	Pipeline stages
n	ALU/branch	IF ID EX MEM WB
n + 4	Load/store	IF ID EX MEM WB
n + 8	ALU/branch	… one cycle later
n + 12	Load/store	… one cycle later

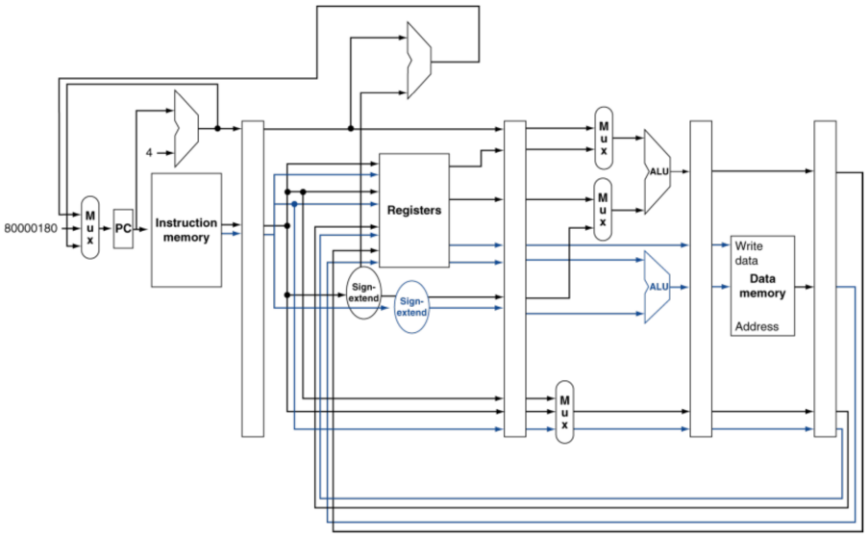


Figure 17.1 — MIPS with static dual issue (slide 93)

Hazards in dual issue (slide 94):

- **EX data hazard:** an ALU result **cannot be used by the load/store in the same packet** (e.g. add $t0,$s0,$s1 with lw $s2,0($t0)) → they must go in different packets — effectively a stall.
- **Load-use hazard:** still a one-cycle use latency, but now **two** instructions' worth of slots are lost → more aggressive scheduling is needed.

4. Worked Example: Scheduling the Array Loop (slide 95)

```
Loop: lw   $t0, 0($s1)        # $t0 = array element
      addu $t0, $t0, $s2      # add scalar in $s2
      sw   $t0, 0($s1)        # store result
      addi $s1, $s1, -4       # decrement pointer
      bne  $s1, $zero, Loop   # branch if $s1 != 0
```

Reasoning, slot by slot:

1. lw must come first (everything depends on it) → **cycle 1** load/store slot.
2. addu needs the loaded value → load-use latency → earliest **cycle 3**.
3. addi is independent → fill the ALU slot of **cycle 2**.
4. sw needs addu's result (different packet) → **cycle 4**; bne goes in cycle 4's ALU slot.
5. Since addi already decremented $s1, the store uses **4($s1)**.

	ALU/branch	Load/store	Cycle
Loop:	nop	lw $t0, 0($s1)	1
	addi $s1, $s1, –4	nop	2
	addu $t0, $t0, $s2	nop	3
	bne $s1, $zero, Loop	sw $t0, 4($s1)	4

IPC = 5 / 4 = **1.25** (peak IPC = 2)

5. Loop Unrolling (slides 96–97)

Replicate the loop body to expose more parallelism and **reduce loop-control overhead** (one addi and one bne per four elements). Use **different registers** for each copy — **register renaming** — to avoid loop-carried **anti-dependences** (a later copy reusing a register an earlier copy still needs; also called **name dependence**).

Unrolled four times:

	ALU/branch	Load/store	Cycle
Loop:	addi $s1, $s1, –16	lw $t0, 0($s1)	1
	nop	lw $t1, 12($s1)	2
	addu $t0, $t0, $s2	lw $t2, 8($s1)	3
	addu $t1, $t1, $s2	lw $t3, 4($s1)	4
	addu $t2, $t2, $s2	sw $t0, 16($s1)	5
	addu $t3, $t3, $s2	sw $t1, 12($s1)	6
	nop	sw $t2, 8($s1)	7
	bne $s1, $zero, Loop	sw $t3, 4($s1)	8

IPC = 14 / 8 = **1.75** (closer to 2, at the cost of registers and code size)

NOTE

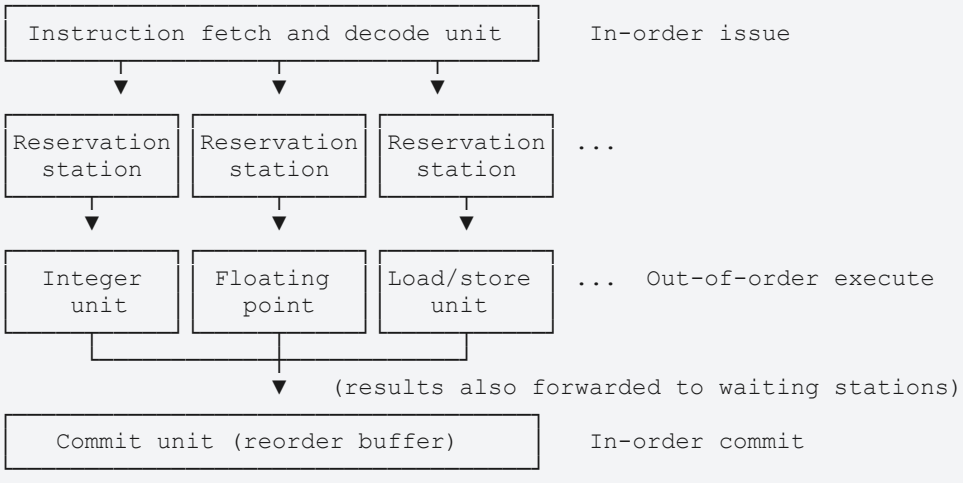
- In cycle 1, lw reads the **old** $s1 (instructions in the same packet do not see each other's results), so the later loads use offsets 12, 8, 4 from the **new** $s1 = old − 16.
- The slide prints "addu $t3, **$t4**, $s2" — a well-known typo; it should be **addu $t3, $t3, $s2**.

6. Dynamic Multiple Issue and Out-of-Order Execution (slides 98–99)

A **superscalar** CPU decides each cycle whether to issue 0, 1, 2, … instructions, avoiding structural and data hazards itself. **Dynamic pipeline scheduling** lets the CPU **execute instructions out of order** to avoid stalls, but **commit results to registers in order**.

```
lw   $t0, 20($s2)
addu $t1, $t0, $t2     # must wait for lw
sub  $s4, $s4, $t3     # independent → can start while addu waits
slti $t5, $s4, 20
```

Block diagram (2021-22 Q2(b)) — textbook organisation; the slides give only the idea:



Reservation stations hold an instruction with its operands until it can execute; the reorder buffer holds results until they can be committed in program order (this also makes speculation and precise exceptions possible).

Exam Approach — Dual-Issue Scheduling

1. Label each instruction **ALU/branch** or **load/store**; count both kinds → a lower bound on cycles.

2. Put lw first; place each user of a load **at least 2 cycles later**.
3. An ALU result may be used by the load/store **in the next packet**, never in the same packet.
4. Fill empty ALU slots with **independent** instructions (pointer updates); fix offsets of loads/stores that move below the pointer update.
5. bne goes in the last packet. **IPC = instructions per iteration ÷ cycles per iteration** (do not count nops).
6. "Technique to increase IPC?" → **loop unrolling with register renaming** (plus scheduling).

TRAP

- Forgetting to adjust the offset (0 → 4) when sw moves after addi.
- Counting nops as instructions in the IPC.
- Putting an instruction and the one that uses its result in the **same** packet.

Practice — Lesson 17

Part (a): Actual — 2023-24 Q4(b) and 2021-22 Q2(c). Part (b): Actual — 2020-21 Q5. Part (c): Constructed (slide 86).

(a) Schedule the array loop of §4 for dual-issue MIPS (one ALU/branch + one load/store, ALU/branch first). What is the IPC? Which technique can further increase the IPC, and how?

(b) Refactor the loop below for multiple issue so that IPC is maximised, and compute the effective IPC.

```
Loop: lw   $t0, 0($s0)
      lw   $t1, 0($s1)
      addu $t0, $t0, $t1
      sw   $t0, 0($s1)
      addi $s0, $s0, -4
      addi $s1, $s1, -4
      bne  $s1, $zero, Loop
```

(c) A 3 GHz processor can issue 4 instructions per cycle. What are its peak instruction rate, peak IPC and peak CPI? Why is the real IPC lower?

✓ Solution

(a) The schedule in §4: 4 cycles for 5 instructions → **IPC = 1.25**. **Loop unrolling** (with register renaming) increases it: unroll 4 times, use $t0–$t3 for the copies, one addi $s1,$s1,–16 and one bne per 4 elements, and interleave loads, adds and stores as in §5 → 14 instructions in 8 cycles → **IPC = 1.75**.

(b) On the dual-issue MIPS (3 load/store + 4 ALU/branch instructions):

	ALU/branch	Load/store	Cycle
Loop:	addi $s0, $s0, –4	lw $t0, 0($s0)	1
	addi $s1, $s1, –4	lw $t1, 0($s1)	2
	nop	nop	3
	addu $t0, $t0, $t1	nop	4
	bne $s1, $zero, Loop	sw $t0, 4($s1)	5

Each lw reads the old pointer (same packet as its addi); addu waits for the load-use latency of $t1; sw uses offset 4 because $s1 was already decremented. **IPC = 7 / 5 = 1.4.**

Going beyond one copy (the hint allows it): **unroll twice** with renamed registers:

	ALU/branch	Load/store	Cycle
Loop:	addi $s0, $s0, –8	lw $t0, 0($s0)	1
	addi $s1, $s1, –8	lw $t1, 0($s1)	2
	nop	lw $t2, 4($s0)	3
	addu $t0, $t0, $t1	lw $t3, 4($s1)	4
	nop	sw $t0, 8($s1)	5
	addu $t2, $t2, $t3	nop	6
	bne $s1, $zero, Loop	sw $t2, 4($s1)	7

11 instructions in 7 cycles → **IPC = 11/7 ≈ 1.57**, and per element the loop now costs 3.5 cycles instead of 5. More unrolling (or wider issue) approaches the peak of 2.

(c) Peak rate = 3 GHz × 4 = **12 BIPS**; peak **IPC = 4**; peak **CPI = 0.25**. Real IPC is lower because dependences (data hazards, load-use latency, branches) leave issue slots empty.

Lesson 18 — Exceptions in the Pipeline (not in the slide file)

Asked in 2022-23 (CSE 209) Q8 and 2018-19 Q2(c). The provided slide file stops at slide 99 and has no exception slides — this short kit follows the textbook (§4.9).

> **NOTE** Use this only as a compact answer kit. If Rifat Sir's full deck has exception slides, follow their figure and signal names.

1. Key Ideas

- **Exception:** an unexpected event from inside the processor (arithmetic **overflow**, **undefined instruction**, TLB/page fault). **Interrupt:** the event comes from outside (I/O).
- MIPS response: save the address of the offending instruction (MIPS actually saves that address + 4) in the **EPC** (Exception Program Counter), record the reason in the **Cause** register, and jump to the handler at **8000 0180$_{hex}$** (this constant already appears at the PC mux in slide 93).
- In a pipeline, an exception is **another control hazard**.
- **Precise exception:** every instruction **before** the offending one completes; the offending instruction and everything **after** it leave **no effect**; EPC identifies the offending instruction so it can be restarted.

2. Hardware Changes for Precise Exceptions

1. Add **EPC** and **Cause** registers (written when an exception is detected).
2. Add a third input **8000 0180$_{hex}$** to the PC-source mux.
3. Add flush controls: **IF.Flush** (zero IF/ID), **ID.Flush** (mux zeroing the control fields entering ID/EX) and **EX.Flush** (mux zeroing the control fields entering EX/MEM) — so the offending instruction and the ones behind it become nops, and the offending instruction cannot write its result (e.g. RegWrite of an overflowing add is cancelled).
4. Detection inputs: **overflow** from the ALU (EX) and **undefined opcode** from the control unit (ID).

Example — overflow in add (in EX): flush the add and the two instructions behind it (in ID and IF), let the instructions ahead in MEM/WB finish, save the add's address (+4 in MIPS) in EPC, set Cause, and fetch from 8000 0180$_{hex}$ in the next cycle.

TLB miss (2018-19 Q2(c)): raised in IF (instruction fetch) or MEM (data access); handled the same way — flush, save EPC/Cause, run the handler that loads the page-table entry into the TLB, then **restart** the instruction at EPC.

Quick Notes — All Lessons

Every lesson's revision points in one place. Figure numbers point back to the lessons.

L9 · Pipelining Basics

- **Stages:** IF (fetch, PC + 4) · ID (decode, read registers) · EX (ALU / address) · MEM (data memory) · WB (write register).
- **Single-cycle clock** = sum of the longest instruction's stage delays (lw). **Pipelined clock** = slowest stage.
- Pipelining improves **throughput**, not latency (each instruction still takes 5 stages — often longer than single-cycle).

$$\text{Pipelined cycles} = 5 + (n - 1) + \text{stalls} \qquad \text{Speedup} = T_{single} / T_{pipelined}$$

$$\text{Balanced stages: time between instructions}_{pipelined} = \text{time between instructions}_{nonpipelined} / \text{number of stages}$$

- Slide example (200/100/200/200/100 ps): 3 lw → 2400 vs 1400 ps → **1.714**; +1,000,000 instructions → **≈ 4** (= 800/200, not 5: unbalanced stages).
- 2017-18 variant (ALU 100 ps): single-cycle 700 ps, 3 lw → 2100 vs 1400 → **1.5**; +3,000,000 → **≈ 3.5**.
- MIPS suits pipelining: fixed 32-bit instructions, few regular formats, load/store addressing (address in EX, access in MEM), aligned operands.

L10 · Pipeline Registers and Control

Register	Basic contents	Bits
IF/ID	instruction, PC + 4	64
ID/EX	PC + 4, Read data 1, Read data 2, sign-extended imm	128
EX/MEM	branch target, Zero, ALU result, Read data 2	97
MEM/WB	memory data, ALU result	64

- Plus register numbers (rt, rd → destination; rs with forwarding) and control bits as needed (147 / 107 / 71 with the WB/M/EX fields).
- **Destination register number must travel to WB** (otherwise the wrong register is written).
- Control generated in ID, carried along: **EX** — RegDst, ALUOp, ALUSrc · **MEM** — Branch, MemRead, MemWrite · **WB** — RegWrite, MemtoReg. ID/EX carries WB+M+EX, EX/MEM carries WB+M, MEM/WB carries WB.
- Multi-cycle diagram = whole program over time; single-cycle diagram = snapshot of one cycle.

L11 · Hazards without Forwarding

- **Structural:** resource conflict (single memory) → separate I/D memories or caches.
- **Data:** RAW dependence on an unfinished result. **Control:** next fetch depends on a branch.
- Register file: **write first half, read second half**.
- No forwarding: consumer **d** instructions after producer → **max(0, 3 − d)** stalls (adjacent → 2).
- sub $2 / and $2,$2 / or $6,$2 with no forwarding → 4 stalls, **11 cycles**; with forwarding → **7 cycles**.

L12 · Forwarding

```
EX hazard:   if (EX/MEM.RegWrite and EX/MEM.RegisterRd ≠ 0
                 and EX/MEM.RegisterRd = ID/EX.RegisterRs)  ForwardA = 10
             (same with RegisterRt → ForwardB = 10)
MEM hazard:  if (MEM/WB.RegWrite and MEM/WB.RegisterRd ≠ 0
                 and not (EX hazard for the same operand)
                 and MEM/WB.RegisterRd = ID/EX.RegisterRs)  ForwardA = 01
             (same with RegisterRt → ForwardB = 01)
```

- Mux codes: **00** register file (ID/EX) · **10** EX/MEM ALU result · **01** MEM/WB write-back value.
- Forwarding unit inputs: ID/EX.Rs, ID/EX.Rt, EX/MEM.Rd, EX/MEM.RegWrite, MEM/WB.Rd, MEM/WB.RegWrite; outputs ForwardA, ForwardB.
- **Double data hazard:** add $1,$1,$2; add $1,$1,$3; add $1,$1,$4 — both EX/MEM and MEM/WB match → take the **most recent** (EX/MEM). That is why the MEM condition has "and not EX hazard".
- "RegisterRd" = destination chosen by RegDst (rt for lw/addi).
- Forwarding cannot solve a **load followed by its use** (Figures 12.1–12.5).

L13 · Load-Use Hazard and Stalls

```
if (ID/EX.MemRead and
    (ID/EX.RegisterRt = IF/ID.RegisterRs or ID/EX.RegisterRt = IF/ID.RegisterRt))
    stall
```

- **How to stall:** zero the control fields entering ID/EX (bubble = nop through EX, MEM, WB); **PCWrite = 0** and **IF/IDWrite = 0** (re-decode the user, re-fetch the next); then forward from MEM/WB.
- HDU inputs: ID/EX.MemRead, ID/EX.Rt, IF/ID.Rs, IF/ID.Rt. Outputs: PCWrite, IF/IDWrite, control-mux select.
- lw → sw of the same value can be forwarded into MEM (slide 66) — the standard HDU still stalls.
- 2022-23 Q2 code (lw/addi/lw/sw): lw→addi stall, addi→lw forward, lw→sw stall (standard HDU) → **10 cycles**.

L14 · Cycle Counting and Scheduling

- With forwarding: ALU → use = 0 stalls; **lw → next instruction = 1 stall**; lw two away = 0.
- Cycles = 5 + (n − 1) + stalls.
- Slide-22 / A = B + C; D = A + E code: **13 cycles → 11** by hoisting the third load.
- Legal reordering: keep RAW order; never move a load above a store to the **same address**.
- Code reordering ↔ "performance via pipelining".

L15 · Branch Hazards

- Branch decided in **MEM** → flush 3 → **3-cycle** penalty. Decided in **ID** (target adder + comparator moved to ID) → **1-cycle** penalty.
- Options: stall on branch · **predict not taken** (flush 1 if taken; IF.Flush zeroes IF/ID) · delayed branch.
- **Four ID additions:** shift-left-2, branch-target adder, register equality comparator, IF.Flush logic.
- **Branch data hazards (compare in ID):** 2nd/3rd preceding ALU → 0 stalls; immediately preceding ALU or 2nd preceding load → **1**; immediately preceding load → **2**.
- **Flush + stall together** (branch in MEM taken while a load-use stall is requested): stall would block the PC from loading the target → give flush priority: stall = load-use AND NOT PCSrc.
- **Jump:** decoded in ID → 1 stall (Chandler right); control in IF lengthens the clock (Joey's problem); BTB indexed by PC in IF avoids it most of the time (Ross).
- 2023-24 Q1(b): both NT **15**, first T **12**, first NT + second T **14** cycles.

L16 · Branch Prediction

- Static: backward taken, forward not taken. Dynamic: **branch history table** indexed by branch address; on a miss, flush and flip.
- **1-bit on nested loops:** 2 mispredictions per inner loop (exit + first iteration next time) → e.g. 80 % for 10 iterations. **2-bit:** change after two successive mispredictions → 90 %.
- 2-bit trace for T, T, NT, T from strong NT: first pass **0 %**, steady state **75 %** (1-bit: 50 %).
- **Perfect predictor still loses 1 cycle** on a taken branch (target known only in ID) → **Branch Target Buffer** indexed by PC in IF.
- 2-bit FSM: Figure 16.1.

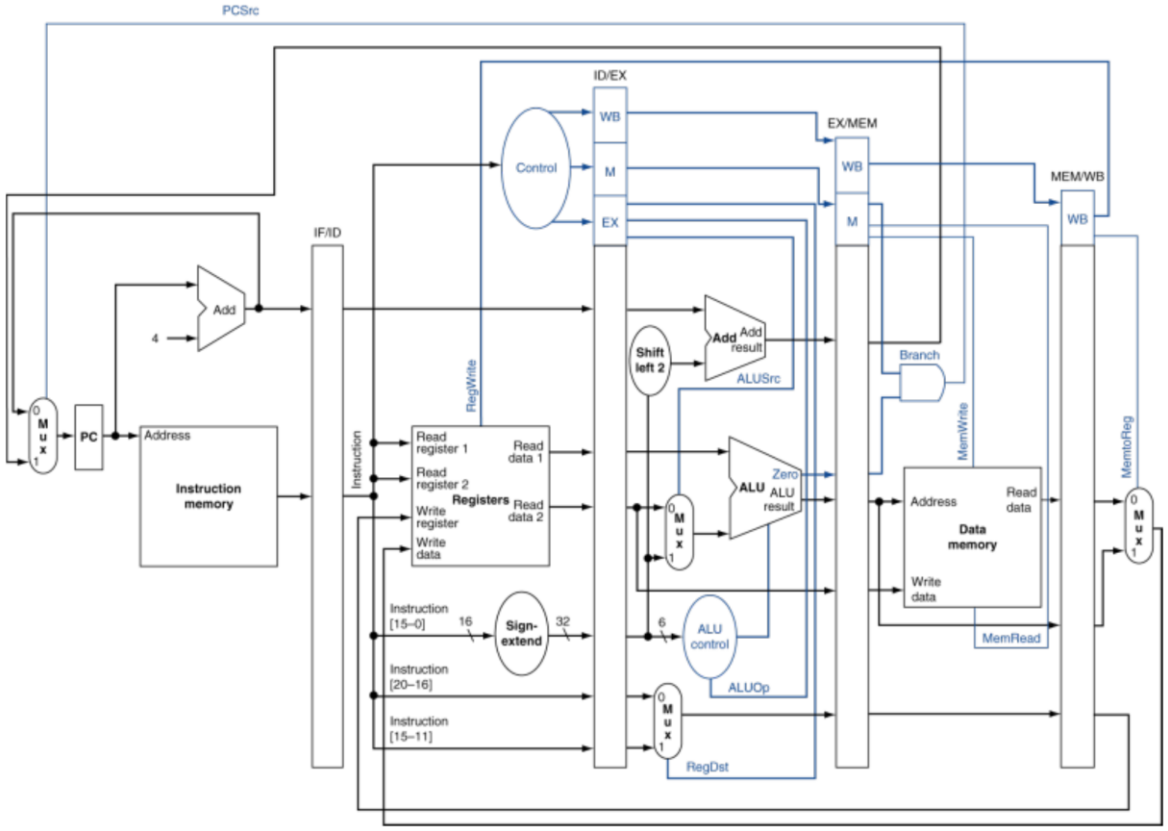
L17 · Multiple Issue

- ILP: deeper pipeline or multiple issue (IPC > 1). 4 GHz 4-way → 16 BIPS, CPI 0.25, IPC 4 (peak).
- **Static (VLIW)**: compiler forms issue packets, removes hazards, pads nops. **Dynamic (superscalar)**: CPU issues 0…n per cycle, may execute **out of order, commit in order**.
- **Speculation:** guess, start early, check, roll back if wrong (branch outcome, loads).
- MIPS dual issue: **ALU/branch + load/store** per 64-bit packet. ALU result cannot feed a load/store in the **same** packet; load-use latency still 1 cycle.
- Array loop: **IPC 5/4 = 1.25**; unrolled ×4 with register renaming: **14/8 = 1.75**. Adjust offsets after the pointer update (sw 4($s1)).
- 2020-21 loop (two loads): dual-issue 5 cycles → **IPC 1.4**; unrolled ×2 → 11/7 ≈ **1.57**.
- Superscalar block diagram: fetch/decode (in-order issue) → reservation stations → functional units (out-of-order execute) → commit unit / reorder buffer (in-order commit).

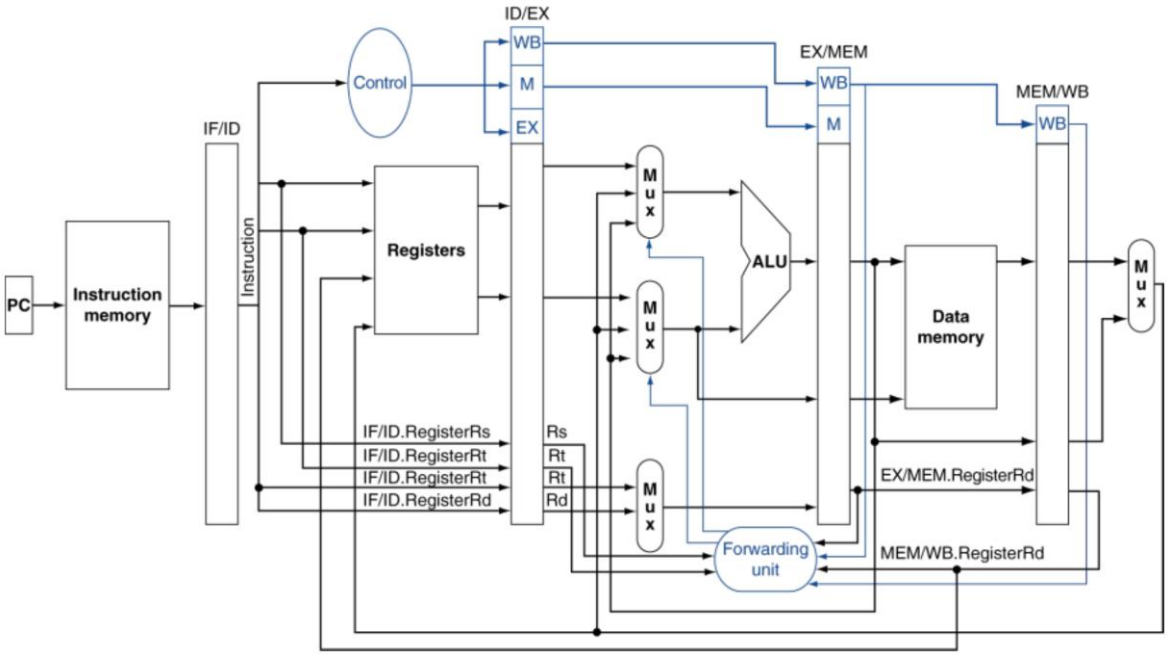
L18 · Exceptions (not in the slide file)

- EPC + Cause registers; PC mux input 8000 0180_{hex}; IF.Flush, ID.Flush, EX.Flush.
- Precise: earlier instructions complete, offending and later ones have no effect, EPC identifies the offender for restart.

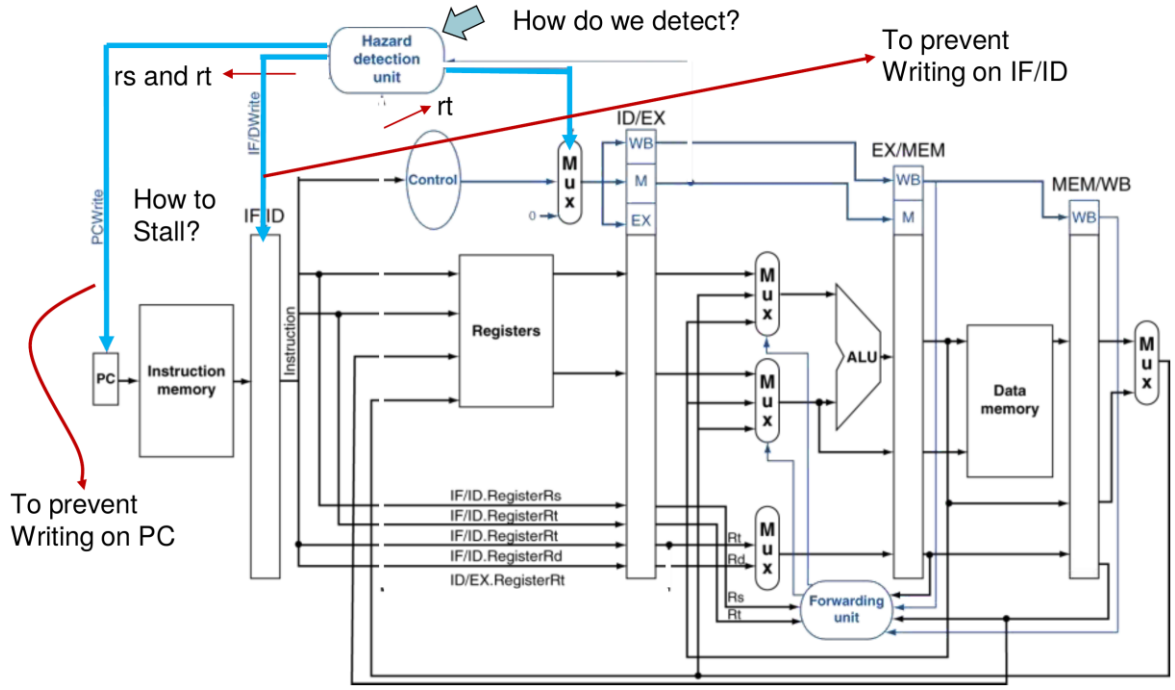
Key Diagrams to Reproduce



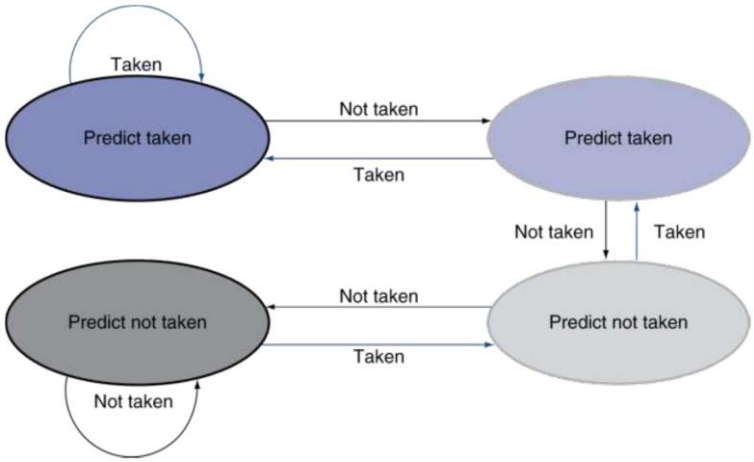
Pipelined datapath with control (slide 52) — same as Fig. 10.6



Datapath with forwarding (slide 65) — same as Fig. 12.5



Datapath with hazard detection (slide 75) — same as Fig. 13.3



2-bit branch predictor (slide 84) — same as Fig. 16.1